RKNN3 Toolkit Lite Python API 参考手册

文件标识：RK-KF-YF-432

发布版本：V1.2.0

日期：2026-10-23

文件密级：□绝密 □秘密 □内部资料 ■公开

免责声明

本文档按“现状”提供，瑞芯微电子股份有限公司（“本公司”，下同）不对本文档的任何陈述、信息和内容的准确性、可靠性、完整性、适销性、特定目的性和非侵权性提供任何明示或暗示的声明或保证。本文档仅作为使用指导的参考。

由于产品版本升级或其他原因，本文档将可能在未经任何通知的情况下，不定期进行更新或修改。

商标声明

“Rockchip”、“瑞芯微”、“瑞芯”均为本公司的注册商标，归本公司所有。

本文档可能提及的其他所有注册商标或商标，由其各自拥有者所有。

版权所有 © 2025 瑞芯微电子股份有限公司

超越合理使用范畴，非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

瑞芯微电子股份有限公司

Rockchip Electronics Co., Ltd.

地址： 福建省福州市铜盘路软件园A区18号

网址： www.rock-chips.com

客户服务电话： +86-4007-700-590

客户服务传真： +86-591-83951833

客户服务邮箱： fae@rock-chips.com

前言

概述

本文是 RKNN3 Toolkit Lite Python API参考手册。

读者对象

本文档（本指南）主要适用于以下工程师：

技术支持工程师

软件开发工程师

修订记录

版本	修改人	修改日期	修改说明	核定人
V0.5.0	HPC团队	2025-12-05	初始版本	熊伟
V1.0.0	HPC团队	2026-01-05	完善 API 说明	熊伟
V1.0.4	HPC团队	2026-05-06	1. 增加 input_callback；2. 增加 LLM 推理暂停与恢复	熊伟
V1.1.0	HPC团队	2026-08-06	1. 补充加密模型加载接口说明；2. 补充多 Session 相关接口说明； 3. 补充动态 Shape 查询接口说明；4. 补充 LoRA / KV Cache 相关接口说明；5. 补充异步会话控制接口说明	熊伟
V1.2.0	HPC团队	2026-10-23	1. 新增分页 KV Cache、Grammar 受约束生成接口说明； 2. 补充 load_rknn 的 source 参数及 keep_grammar 推理参数说明	熊伟

目录

1. API 参考说明

   1.1 适用版本和平台

   1.2 安装与导入

   1.3 基本使用流程

      通用模型

      LLM/VLM

   1.4 Python 类型和 C 类型的对应关系

   1.5 异常处理约定

2. RKNN3Lite 编程模型

   2.1 对象生命周期

   2.2 模型、Runtime 和 Session 的关系

   2.3 同步和异步执行

      通用模型

      LLM/VLM

   2.4 内存所有权和释放

      高层输入和输出

Runtime 创建的 Tensor Memory
用户提供的模型运行内存
Output Callback 内存
2.5 线程与 Callback 生命周期
3. RKNN3Lite 类
3.1 对象初始化和释放
init
release
3.2 模型加载与 Runtime 初始化
load_rknn
init_runtime
3.3 通用模型推理
inference
3.4 异步推理与等待
inference_async
wait
get_outputs
3.5 张量属性和动态 Shape
get_inputs_tensor_attr
get_dynamic_shape_infos
get_outputs_tensor_attr
set_shape
3.6 张量与内存管理
create_mem
create_mem_from_phys
create_mem_from_fd
destroy_mem
mem_sync
mem_sync_range
set_internal_mem
set_weight_mem
set_input_name_alias
3.7 LLM Session 生命周期

init_llm_session
get_session_count
3.8 LLM 推理控制
session_run
session_run_async
session_stop
session_pause
session_resume
session_query_state
set_llm_param
3.9 Chat Template、Function Calling 和 Grammar
set_chat_template
set_function_tools
init_grammar
set_grammar
clear_grammar
3.10 KVCache 管理
set_kvcache_policy
load_kvcache
save_kvcache
clear_kvcache
set_kvcache_mem
set_kvcache_group
set_kvcache_pages
3.11 LoRA 管理
init_lora
load_lora
unload_lora
enable_lora
disable_lora
query_lora
3.12 Callback 配置
Callback 配置示例
create_output_tensors

3.13 Profile 和调试

dump_features

profile_ops

profile_mem

3.14 图像内存与图像处理

im_mem_create

im_mem_destroy

im_cvt_color

3.15 设备查询和上下文复制

get_sdk_version

get_devices_id

dup_context

rknn3_query

4. RKNN3 类型定义

4.1 常量、状态码和枚举

4.1.1 常量定义

4.1.2 状态码

4.1.3 RKNN3QueryCmd

4.1.4 RKNN3TensorType

4.1.5 RKNN3TensorQntType

4.1.6 RKNN3TensorLayout

4.1.7 RKNN3MemAllocFlags

4.1.8 RKNN3MemSyncMode

4.1.9 RKNN3CoreMask

4.1.10 RKNN3KVCachePolicy

4.1.11 RKNN3KVCacheClearPolicy

4.1.12 RKNN3LLMInputType

4.1.13 LLMCallState

4.1.14 RKNN3GrammarType

4.1.15 其他枚举类型

4.2 通用结构

4.2.1 RKNN3InputOutputNum

4.2.2 RKNN3PerfDetail

4.2.3 RKNN3PerfRun

4.2.4 RKNN3SDKVersion
4.2.5 RKNN3CustomString
4.2.6 RKNN3Config
4.2.7 RKNN3ShapeInfo
4.2.8 RKNN3ShapeConfig
4.2.9 RKNN3InitExtend
4.2.10 RKNN3NodeMemInfo
4.2.11 RKNN3DevMemInfo
4.2.12 RKNN3Device
4.2.13 RKNN3Devices
4.2.14 自定义算子结构
4.3 张量与内存结构
4.3.1 RKNN3QuantInfo
4.3.2 RKNN3TensorAttr
4.3.3 RKNN3MemSize
4.3.4 RKNN3TensorMemory
4.3.5 RKNN3Tensor
4.3.6 RKNN3AllocationInfo
4.3.7 RKNN3CoreMemSize
4.4 LLM 和 VLM 结构
4.4.1 Float16
4.4.2 RKNN3VocabInfo
4.4.3 RKNN3SamplingParams
4.4.4 RKNN3LLMTensor
4.4.5 RKNN3AuxTensor
4.4.6 RKNN3LLMMultiModelTensor
4.4.7 RKNN3LLMInputUnion
4.4.8 RKNN3LLMInput
4.4.9 RKNN3LLMExtendParam
4.4.10 RKNN3LLMParam
4.4.11 RKNN3LLMInferParam
4.4.12 RKLLMResult
4.4.13 RKLLMRunState
4.4.14 RKLLMResultLastHiddenLayer
4.4.15 RKNN3LLMConfig
4.4.16 RKNN3GrammarConfig

4.4.17 RKNN3GrammarSetOptions

4.5 KVCache 结构

4.5.1 RECURRENT

4.5.2 RKNN3KVCachePolicyParam

4.5.3 RKNN3KVCacheClearPolicyParam

4.5.4 RKNN3AttentionKvCacheLens

4.5.5 RKNN3KvCacheLenGroupInfo

4.5.6 RKNN3AttentionConfig

4.5.7 RKNN3CachePageAttr

4.5.8 RKNN3KvCachePageInfo

4.5.9 RKNN3KvCachePageBinding

4.6 LoRA 结构

4.6.1 RKNN3Lora

4.7 图像、音频和视频结构

4.7.1 RKNN3Image

4.7.2 RKNN3Audio

4.7.3 RKNN3Video

4.7.4 RKNN3ImRect

4.7.5 RKNN3ImMetadata

4.7.6 RKNN3ImMem

4.8 Callback 类型

4.8.1 LLMResultCallback

4.8.2 LLMSamplingCallback

4.8.3 LLMGetEmbedCallback

4.8.4 LLMTokenizerCallback

4.8.5 LLMOutputCallback

4.8.6 LLMInputCallback

4.8.7 RKLLMCallback

4.8.8 LLMInputCallbackParam

4.9 包装类

4.9.1 RKNN3AuxTensorWrapper

5. 公共工具函数

5.1 rknn_dtype_to_numpy_dtype

5.2 rknn3_get_layout_string
5.3 rknn3_get_type_string
5.4 rknn3_get_qnt_type_string
5.5 dump_tensor_attr
5.6 get_os_platform
6. 异常与错误处理
6.1 错误返回与日志
6.2 Runtime 和模型初始化错误
6.3 Query、输入和推理错误
6.3.1 Query 失败
6.3.2 通用模型输入错误
6.4 LLM Session 和输入错误
6.4.1 Session 索引错误
6.4.2 LLM 主输入互斥
6.4.3 上下文长度错误
6.5 KVCache 和 LoRA 错误
6.5.1 KVCache
6.5.2 LoRA
6.6 内存相关错误
6.6.1 内存不足
6.6.2 内存同步失败
6.6.3 外部内存生命周期
6.7 Callback 异常
6.8 异步执行错误
通用模型
LLM
6.9 错误定位建议

1. API 参考说明

本手册用于说明 RKNN3 Toolkit Lite 的 Python API。内容以 `RKNN3Lite` 公开接口和数据类型定义为主体，重点描述接口的调用顺序、参数含义、返回值、资源生命周期以及 LLM/VLM 场景下的 Runtime、Session、Callback 和内存关系。

RKNN3 Toolkit Lite 用于在 Rockchip NPU 平台上加载和运行 RKNN 模型。模型转换由《RKNN3_Toolkit_Python_API_参考》介绍，本手册不涉及模型转换流程。

1.1 适用版本和平台

本文档当前发布版本为 V1.2.0，适用于配套版本的 RKNN3 Toolkit Lite Python 包和 RKNN3 Runtime。

当前支持的平台如下：

项目	支持范围
硬件平台	RK1820/RK1828/RK3572
操作系统	Debian 10/11/12（aarch64）
Python	3.9/3.10/3.11/3.12
主要 Python 依赖	`numpy`、`transformers`、`jinja2`

不同硬件平台和 Runtime 版本或模型类型支持的能力可能存在差异。若具体接口章节给出了额外的平台限制或使用条件，应以对应接口说明为准。

RKNN3 Toolkit Lite 运行依赖设备侧 RKNN3 Runtime 动态库。Python 包会从预定义路径加载相应 Runtime，因此 Python Wheel、设备 Runtime 和 RKNN 模型需要保持版本兼容。

1.2 安装与导入

请通过 `pip3 install` 命令安装 RKNN3-Toolkit Lite。安装步骤如下：

- 如果系统中没有安装 `python3/pip3` 等程序，请先通过 `apt-get` 方式安装，参考命令如下：

```
sudo apt-get update
sudo apt-get install -y python3 python3-dev python3-pip gcc
```

注：部分依赖模块需要编译源码，此时将用到 `python3-dev` 和 `gcc`，此步骤将同时安装这两个包，以避免在安装依赖时编译失败。

- 安装依赖模块：`opencv-python` 和 `numpy`，参考命令如下：

```
sudo apt-get install -y python3-opencv
sudo apt-get install -y python3-numpy
```

或者

```
pip3 install opencv-python
pip3 install numpy
```

注：

1. RKNN3-Toolkit Lite 本身并不依赖 `opencv-python`，但是在示例中需要使用该模块对图像进行处理。

2. 在 Debian 10 固件上通过 `pip3` 安装 `numpy` 可能失败，建议用上述方法安装。

3. 安装 RKNN3-Toolkit Lite

各平台的安装包都放在 SDK 的 `rknn3-toolkit-lite/packages` 文件夹下。进入该文件夹后，执行以下命令来安装 RKNN3-Toolkit Lite：

```
# Python 3.9
pip3 install rknn3_toolkit_lite-x.y.z-cp39-cp39-linux_aarch64.whl
# Python 3.10
pip3 install rknn3_toolkit_lite-x.y.z-cp310-cp310-linux_aarch64.whl
# Python 3.11
pip3 install rknn3_toolkit_lite-x.y.z-cp311-cp311-linux_aarch64.whl
# Python 3.12
pip3 install rknn3_toolkit_lite-x.y.z-cp312-cp312-linux_aarch64.whl
```

`RKNN3Lite` 以及常用 LLM/VLM 类型可以从 `rknn3lite.api` 导入：

```
from rknn3lite.api import (
    RKNN3Lite,
    RKLLMCallback,
    RKNN3Image,
    RKNN3AuxTensorWrapper,
)
```

枚举、结构体和底层类型定义可从 `rknn3_types` 导入：

```
from rknn3lite.api.rknn3_types import (
    RKNN3QueryCmd,
    RKNN3TensorType,
    RKNN3TensorLayout,
    RKNN3KVCachePolicy,
    RKNN3KVCacheClearPolicy,
    RKNN3KVCacheClearPolicyParam,
)
```

普通模型使用：

```
rknn = RKNN3Lite()
```

LLM/VLM 模型使用：

```
rknn = RKNN3Lite(llm_mode=True)
```

`llm_mode=True` 表示当前对象会初始化 LLM Session 相关代码。它不会在构造 `RKNN3Lite` 对象时立即创建 Runtime 或 Session，底层资源仍由后续的 `init_runtime()` 和 `init_llm_session()` 创建。

1.3 基本使用流程

RKNN3 Toolkit Lite 的调用流程根据模型类型分为通用模型和 LLM/VLM 两类。

通用模型

同步推理的基本流程为：

```
RKNN3Lite()
    ↓
load_rknn()
    ↓
init_runtime()
    ↓
inference()
    ↓
release()
```

基本代码如下：

```
from rknn3lite.api import RKNN3Lite

rknn = RKNN3Lite()

ret = rknn.load_rknn("model.rknn", "model.weight")
if ret != 0:
    raise RuntimeError("load_rknn failed")

ret = rknn.init_runtime(target="rk1820", core_mask=0x01)
if ret != 0:
    raise RuntimeError("init_runtime failed")

outputs = rknn.inference(inputs=[input_data])
if outputs is None:
    raise RuntimeError("inference failed")

rknn.release()
```

通用模型也支持异步执行：

```
inference_async()
    ↓
wait()
    ↓
get_outputs()
```

LLM/VLM

LLM/VLM 推荐使用“先初始化 Runtime、查询模型配置、再初始化 Session”的分步流程：

```
RKNN3Lite(llm_mode=True)
    ↓
load_rknn()
    ↓
init_runtime()
    ↓
rknn3_query(RKNN3_QUERY_LLM_CONFIG)
    ↓
构造 llm_args 和 RKLLMCallback
    ↓
init_llm_session()
    ↓
session_run()/session_run_async()
    ↓
release()
```

采用分步初始化后，可以在创建 Session 前读取模型实际的 `vocab_size`、`embedding_dim`、`max_ctx_len`、`max_position_embeddings`、KVCache 配置和任务类型，避免在应用中重复硬编码模型参数。

如果应用已经提前确定 `llm_args` 和 Callback，也可以在 `init_runtime()` 时直接传入相关参数，由初始化流程创建首个 LLM Session。

一个 `RKNN3Lite` LLM 对象可以在同一 Runtime 上创建多个 Session。新增 Session 时，应使用连续的 `session_index` 进行管理，各 Session 可以独立维护对话状态、KVCache 以及按 Session 生效的配置。

1.4 Python 类型和 C 类型的对应关系

RKNN3 Toolkit Lite 使用 `ctypes` 对 RKNN3 C API 中的枚举、结构体、指针和回调函数进行封装。Python API 会进一步把部分底层对象转换为 Python 原生类型或 NumPy 数组。

常见类型关系如下：

Python 表示	C 侧表示	说明
`int`	`int32_t` / `uint32_t` / `uint64_t` 等	实际位宽由具体参数或结构字段决定

Python 表示	C 侧表示	说明
`bool`	`bool` 或整型标志位	具体存储类型由对应 C 字段决定
`str`	`const char *` / `char *`	Python 封装负责字符串编码和传递
`bytes` / `bytearray`	`void *` / `const void *` + size	常用于解密数据、LoRA、KVCache 等二进制数据
`numpy.ndarray`	Tensor 数据缓冲区	高层推理接口负责将 NumPy 数据转换或绑定到 Runtime Tensor
`IntEnum` 子类	C `enum` /整型枚举值	如 `RKNN3QueryCmd`、`RKNN3TensorType`
`ctypes.Structure` 子类	C `struct`	如 `RKNN3TensorAttr`、`RKNN3TensorMemory`、`RKNN3LLMConfig`
`ctypes.POINTER(T)`	`T *`	用于 Tensor、内存、数组等底层指针
`ctypes.c_void_p`	`void *`	常用于 userdata、Session 或通用内存地址
`ctypes.CFUNCTYPE` 回调对象	C 函数指针	用于 LLM Result、Tokenizer、Embedding、Input、Output 等 Callback
`RKNN3TensorAttr`	`rknn3_tensor_attr`	描述 Tensor 的 Shape、dtype、Layout、量化和对齐信息
`RKNN3TensorMemory`	`rknn3_tensor_mem`	描述 Tensor 的虚拟地址、物理地址、FD、大小等内存信息
`RKNN3Tensor`	`rknn3_tensor`	通过 `attr` 和 `mem` 关联 Tensor 属性与实际缓冲区
`RKNN3Lora`	`rknn3_lora`	LoRA Adapter 描述信息
`RKLLMCallback`	LLM Callback 结构	保存各类回调函数、userdata 以及 Input/Output Callback 张量配置

`rknn3_context` 在 Python 层根据运行架构使用无符号整数表示；`rknn3_session` 使用 `c_void_p` 表示。普通应用不需要直接操作这两个底层类型，通常通过 `RKNN3Lite` 和 `session_index` 管理。

需要注意，Python 高层参数与 C 参数并非始终一一对应。例如：

- Python `session_index` 是 Python 封装对多个底层 Session 的管理索引，不是 C Session 指针本身；
- `numpy.ndarray` 是应用侧数据对象，高层接口会根据 Tensor 属性完成必要的数据准备；
- `list`、Python 字典等对象可能在进入 Runtime 前被转换成结构体数组或对应 C 参数。

完整枚举和结构定义见第 4 章。

1.5 异常处理约定

RKNN3 Toolkit Lite Python 封装主要采用“记录错误日志并通过返回值表示失败”的方式处理 Runtime 错误。调用方不应只依赖 Python `try/except`，而应按照具体接口说明检查返回值。

常见返回约定如下：

接口类型	成功时	失败时
状态型接口	`0`	`-1` 或其他非 `0` /负错误码
对象或查询接口	对应对象或查询结果	`None`
列表型接口	数据列表	`None` 或空结果，具体以接口说明为准
复合返回接口	`(ret, data)`、`(data, count)` 等	失败元组，具体以接口说明为准

底层 Runtime 返回码包括参数错误、模型错误、Context 错误、运行失败、内存不足、超时、设备不可用、版本不兼容等。Python 层定义了 `rknn3_runtime_api_ret_code` 用于将对应错误码转换为可读状态，完整状态码见 4.1.2 状态码。

建议采用以下处理方式：

```
ret = rknn.init_runtime(target="rk1820", core_mask=0x01)
if ret != 0:
    # 根据日志和返回码处理错误
    rknn.release()
    raise RuntimeError(f"init_runtime failed, ret={ret}")
```

对于返回对象的接口，应显式检查 `None`：

```
llm_config = rknn.rknn3_query(RKNN3QueryCmd.RKNN3_QUERY_LLM_CONFIG)
if llm_config is None:
    raise RuntimeError("query LLM config failed")
```

Python 参数处理、文件访问、NumPy 操作、Tokenizer、用户自定义 Callback 等位于 RKNN3 Runtime 封装之外的代码仍可能抛出标准 Python 异常。Callback 中如需执行 Python 逻辑，建议在回调内部自行捕获异常并按回调约定返回，避免异常跨越 `ctypes` 回调边界。

2. RKNN3Lite 编程模型

`RKNN3Lite` 是 RKNN3 Toolkit Lite 的 Python 主入口。理解对象、Runtime、模型、Session、内存和 Callback 之间的关系，有助于正确使用第 3 章中的各类接口。

2.1 对象生命周期

创建 `RKNN3Lite` Python 对象时，仅建立 Python 侧封装对象，不会立即创建 RKNN3 Runtime。

通用模型的完整生命周期为：

```
创建 RKNN3Lite 对象
        ↓
load_rknn()
        ↓
init_runtime()
        ↓
执行 inference/query/profile 等操作
        ↓
release()
```

LLM/VLM 的完整生命周期为：

```
创建 RKNN3Lite(llm_mode=True)
        ↓
load_rknn()
        ↓
init_runtime()
        ↓
init_llm_session()
        ↓
执行 session_run/KVCache/LoRA/Session 控制等操作
        ↓
release(session_index=...) 或 release()
```

其中：

- `load_rknn()` 负责记录并检查 RKNN 模型文件和权重文件路径；
- `init_runtime()` 创建底层 RKNN3 Context，并完成模型加载与模型初始化；
- `init_llm_session()` 在已经初始化的 LLM Runtime 上创建 Session；
- `release(session_index=n)` 只释放指定 LLM Session，共享 Runtime 继续有效；
- `release()` 不指定 Session 时，释放当前对象管理的全部 Session 和 Runtime。

资源释放后，与该 Session 或 Runtime 关联的 Tensor、Memory、Callback 数据和底层句柄均不应继续使用。

为了保证异常路径也能够释放资源，应用应在退出路径中调用 `release()`。对于显式创建的 Tensor Memory，还需要按照对应接口要求调用 `destroy_mem()`。

2.2 模型、Runtime 和 Session 的关系

在一个 `RKNN3Lite` 实例内部，可以将几个核心对象理解为以下层次：

```
RKNN3Lite Python 对象
        │
        ├── RKNN 模型文件/Weight 文件
        │
        ▼
RKNN3 Runtime/Context
        │
        ├── 通用模型：直接执行 inference()
        │
        └── LLM/VLM:
              ├── Session 0
              ├── Session 1
              ├── Session 2
              └── ...
```

`load_rknn()` 本身不会完成底层模型初始化。真正的模型加载、Context 创建和 NPU Runtime 初始化发生在 `init_runtime()` 中。

对于通用模型，Runtime 初始化完成后即可执行 `inference()`、`inference_async()`、Tensor Query、动态 Shape、Profile 和显式内存管理等操作，不需要创建 Session。

对于 LLM/VLM，Session 构建在已经初始化的 Runtime/Context 之上。多个 Session 共享同一个底层模型和 Runtime，但每个 Session 拥有独立的会话状态。Chat Template、KVCache、推理状态以及按 Session 生效的 LoRA 等操作通过 `session_index` 指定目标 Session。

多 Session 初始化时，`session_index` 需要连续创建。例如已经存在 Session 0 时，可以继续初始化 Session 1，但不能直接跳到 Session 2。

LLM Session 有两种初始化方式：

直接初始化

```
load_rknn()
    ↓
init_runtime(llm_args=..., llm_callback=...)
    ↓
Session 0 可用
```

分步初始化

```
load_rknn()
    ↓
init_runtime()
    ↓
rknn3_query()
    ↓
init_llm_session()
```

分步初始化适合需要先查询模型配置、再根据模型实际参数构造 `llm_args`、Embedding 和 Callback 的场景，也是多 Session 管理时更清晰的使用方式。

2.3 同步和异步执行

RKNN3 Toolkit Lite 对通用模型和 LLM Session 分别提供同步、异步接口，两套异步流程的完成方式不同。

通用模型

同步调用：

```
inference()
    ↓
方法返回时，本次推理已经完成
```

异步调用：

```
inference_async()
    ↓
wait()
    ↓
get_outputs()
```

`inference_async()` 负责提交任务；`wait()` 用于等待该异步任务完成；完成后通过 `get_outputs()` 获取结果。

LLM/VLM

同步调用：

```
session_run()
    ↓
生成完成、停止或发生错误后返回
```

异步调用：

```
session_run_async()
    ↓
Runtime 持续执行
    ↓
LLM Callback 接收输出和运行状态
```

LLM 异步执行不使用通用模型的 `wait()`。应用应根据结果 Callback 中的终态判断一次异步请求是否结束，常见终态包括：

- `RKLLM_RUN_FINISH`
- `RKLLM_RUN_STOP`
- `RKLLM_RUN_MAX_NEW_TOKEN_REACHED`
- `RKLLM_RUN_ERROR`

执行过程中还可以使用 `session_query_state()` 查询 token 数、KVCache 状态等运行信息，并按需要调用：

- `session_stop()`
- `session_pause()`
- `session_resume()`

`session_run_async()` 表示异步提交和回调驱动的执行方式，不应将“同一 Session 上连续提交多个异步请求”直接等同于多请求并行。需要进行任务隔离或并发管理时，应结合多 Session，并由应用层负责调度和同步。

2.4 内存所有权和释放

RKNN3 Toolkit Lite 同时支持高层 NumPy 输入和显式 Tensor Memory。不同接口创建或接收的内存，其所有权不同。

高层输入和输出

`inference()`、`session_run()` 等高层接口通常接收 `numpy.ndarray`、Prompt、Token 或 Embed 等 Python 对象。同步调用期间，这些输入对象必须保持有效。

对于异步调用，为避免底层任务尚未完成时相关 Python 缓冲区被释放，输入数据及其引用应至少保持到对应异步任务结束。

VLM 场景中，如果将 NumPy 数据地址直接转换为 `Float16 *`、`void *` 等指针写入 `RKNN3Image` 或其他结构，对应 NumPy 数组的生命周期必须覆盖底层指针的整个使用周期。

Runtime 创建的 Tensor Memory

通过以下接口创建的 Runtime Tensor Memory：

- `create_mem()`

- `create_mem_from_phys()`
- `create_mem_from_fd()`

在不再使用时，应通过 `destroy_mem()` 释放 RKNN3 对应的内存对象或句柄。

对于 `create_mem()`，实际 Tensor Buffer 由 RKNN3 Runtime 创建和管理。该接口既支持按 `RKNN3TensorAttr` 分配，也支持按 `size` + `core_id` 分配裸内存，两者的释放方式一致，均通过 `destroy_mem()`。

对于 `create_mem_from_phys()`、`create_mem_from_fd()`，物理内存、FD 或外部缓冲区由应用提供。应用需要保证外部存储在 RKNN3 使用期间始终有效；`destroy_mem()` 释放的是 RKNN3 对该内存的关联对象，不代表应用可以提前销毁外部存储。

用户提供的模型运行内存

以下接口允许应用向 Runtime 提供特定用途的内存：

- `set_internal_mem()`
- `set_weight_mem()`
- `set_kvcache_mem()`
- `set_kvcache_pages()`

这类内存由应用负责准备，其生命周期必须覆盖 Runtime 或对应 Session 对它的使用周期。

其中 `set_kvcache_pages()` 绑定的是 page 表，调用方必须持有所有成功绑定的 page，直到通过 `n_pages = 0` 解绑。解绑之前不得调用 `destroy_mem()` 释放对应 page。

Output Callback 内存

使用 LLM Output Callback 时，可以由应用创建 `RKNN3Tensor` 数组并为每个输出调用 `create_mem()`。这些 Tensor 会通过：

```
RKLLMCallback.output_tensors
RKLLMCallback.n_output_tensors
```

注册给 Session。

回调中读取完成后，不应立即释放仍可能被后续 Callback 使用的内存。Session 使用结束后，再逐个调用 `destroy_mem()`。

调用 `release()` 后，依赖该 Runtime/Session 的底层内存句柄、Tensor 指针和 Session 资源均视为失效。

2.5 线程与 Callback 生命周期

RKNN3 Toolkit Lite 的 Callback 通过 `ctypes` 与底层 Runtime 连接。底层 Runtime 可能在推理执行期间调用 Python Callback，因此 Callback 的 Python 对象生命周期需要由应用显式保证。

需要在 Session 生命周期内保持有效的对象通常包括：

- `LLMResultCallback`
- `LLMSamplingCallback`
- `LLMTokenizerCallback`
- `LLMGetEmbedCallback`
- `LLMInputCallback`
- `LLMOutputCallback`
- `RKLLMCallback`
- 各 Callback 的 userdata
- `ctypes.py_object`
- `input_tensors_index` 数组
- `output_tensors` 数组
- Callback 中直接引用的 NumPy、mmap 或其他外部缓冲区

不应只在局部作用域中创建 Callback 或 userdata，然后在 Session 仍然运行时丢失其 Python 引用。较稳妥的方式是将这些对象保存在长生命周期变量、业务对象成员或专门的 keepalive 容器中。

`LLMInputCallback` 中收到的 Tensor 指针和 `LLMInputCallbackParam` 只用于当前回调过程。应用可以在回调中读取 token、位置和 Tensor 属性，并写入 Runtime 提供的目标缓冲区，但不应保存这些底层指针并在回调返回后继续访问。

`LLMOutputCallback` 中的输出 Tensor 如果来自应用显式创建的 `output_tensors`，其底层内存应保持到最后一次可能使用该输出的 Callback 完成。

Callback 中执行 Python 代码时，建议在回调内部捕获异常并返回约定状态，避免未处理的 Python 异常跨越 `ctypes` 回调边界。

当前 Python 封装和公开接口没有对所有 `RKNN3Lite` 方法给出统一的线程安全保证，因此不应默认同一个对象上的任意接口都可以无条件并发调用。

需要特别区分：

- `session_stop()`、`session_pause()`、`session_resume()` 等接口用于在 Session 正在运行时改变其状态；
- 这不等于同一 Session 上所有 API 都具备并发线程安全保证；
- 同一 Session 的运行、停止、暂停、恢复、释放等操作应由应用统一协调；
- 需要并发隔离时，可使用多个 Session 或独立 Context，并由应用层负责同步。

3. RKNN3Lite 类

RKNN3Lite 是 RKNN3 Toolkit Lite 的主要类，提供模型加载、初始化、推理、LoRA、KVCache 管理、内存管理、异步推理、调试和查询等功能。

3.1 对象初始化和释放

本节控制 Python 对象与底层 Runtime/Session 的生命周期。传统模型创建 `RKNN3Lite()`；LLM/VLM 创建 `RKNN3Lite(llm_mode=True)`。多 Session 场景下，可通过 `release(session_index=...)` 仅释放指定会话，最后再调用不带参数的 `release()` 释放全部 Session 和共享 Runtime。

init

API	`init`
功能	初始化 RKNN3Lite 对象。用于创建 RKNN3Lite 实例，并设置日志级别和文件路径。
参数	`llm_mode`：布尔类型，是否为 LLM 模式，默认 `False`
	`verbose`：布尔类型，是否在终端中打印详细日志，默认 `False`
	`verbose_file`：字符串类型，日志文件路径，如果指定，将日志写入该文件，默认 `None`
返回值	无

示例：

```
from rknn3lite.api.rknn3_lite import RKNN3Lite

# 将详细的日志信息输出到屏幕，并写到inference.log文件中
rknn_lite = RKNN3Lite(verbose=True, verbose_file='./inference.log')

# 初始化传统模型，只在屏幕打印详细的日志信息
rknn_lite = RKNN3Lite(verbose=True)

# 初始化LLM模型
rknn_lite = RKNN3Lite(llm_mode=True, verbose=True)
```

release

API	`release`
功能	释放RKNN资源。若指定 `session_index`，仅释放对应 LLM Session，并保留共享的 RKNN Runtime；若 `session_index` 为 None，则释放全部 LLM Session 和 RKNN Runtime。
参数	`session_index`：可选参数，指定需要释放的 LLM Session 索引；默认值为 None。
返回值	成功：0；失败：-1

示例：

```
# 释放全部资源
ret = rknn_lite.release()

# 仅释放指定 LLM Session
ret = rknn_lite.release(session_index=0)
```

3.2 模型加载与 Runtime 初始化

常见初始化流程如下：

```
传统模型：load_rknn -> init_runtime -> inference
LLM直接初始化：load_rknn -> init_runtime(llm_args, llm_callback) -> session_run
LLM分步初始化：load_rknn -> init_runtime -> rknn3_query -> init_llm_session ->
session_run
```

LLM 推荐采用“分步初始化”流程：先通过 `RKNN3_QUERY_LLM_CONFIG` 获取 `vocab_size`、`max_ctx_len`、`embedding_dim` 等模型信息，再构造 LLM 参数和 Callback。这样可避免把模型相关参数硬编码在应用中。

load_rknn

API	`load_rknn`
功能	设置并校验 RKNN 模型和权重的来源，供后续 `init_runtime()` 加载使用。支持从主机侧文件路径、内存数据或设备侧文件路径三种方式加载。
参数	`model_path`：模型来源，含义由 `source` 决定： - `source="path"` 时为主机侧 `.rknn` 模型文件路径 - `source="data"` 时为内存中的模型数据，支持 `bytes`、`bytearray`、`memoryview` - `source="device"` 时为设备侧文件系统（如 emmc）上的模型文件路径
	`weight_path`：权重来源，含义由 `source` 决定： - `source="path"` 时为主机侧 `.weight` 权重文件路径 - `source="data"` 时为内存中的权重数据，支持 `bytes`、`bytearray`、`memoryview` - `source="device"` 时为设备侧文件系统上的权重文件路径；传 `None` 表示流式权重加载模式
	`source`：模型和权重的来源类型，取值为 `"path"`（默认）、`"data"` 或 `"device"`： - `"path"`：从主机侧文件路径加载，对应 C API `rknn3_load_model_from_path` - `"data"`：从内存数据加载，对应 C API `rknn3_load_model_from_data` - `"device"`：从设备侧文件路径加载，对应 C API `rknn3_load_model_from_device`
返回值	成功：0；失败：-1

示例：

从主机侧文件路径加载（默认方式，与历史版本用法一致）：

```
ret = rknn_lite.load_rknn(model_path='model.rknn', weight_path='model.weight')
if ret != 0:
    print('Load model failed')
```

从设备侧文件路径加载（模型和权重位于设备文件系统上，主机只传递路径）：

```
ret = rknn_lite.load_rknn(model_path='/data/model.rknn',
weight_path='/data/model.weight', source='device')
if ret != 0:
    print('Load model failed')
```

从设备侧文件路径加载模型，权重采用流式加载模式：

```
ret = rknn_lite.load_rknn(model_path='/data/model.rknn', weight_path=None,
source='device')
if ret != 0:
    print('Load model failed')
```

使用说明:

- RKNN 模型与权重需要配套使用。若权重文件与模型同名，可通过 `rknn_path.replace('.rknn', '.weight')` 生成对应权重路径。
- `source="path"` 时，`load_rknn()` 会检查模型和权重文件是否存在；`source="data"` 时会检查数据类型；`source="device"` 时路径指向设备侧文件系统，不做主机侧存在性检查。
- `source="data"` 时，`bytearray` 和 `memoryview` 会在内部转换为 `bytes` 后传递给底层接口，数据在 `init_runtime()` 完成加载前必须保持有效。
- `source="device"` 时，`weight_path` 传 `None` 表示流式权重加载模式，权重数据需在 `init_runtime()` 之后由应用自行分批加载。
- `load_rknn()` 只记录并校验模型/权重来源，真正的 Runtime、模型初始化和 NPU 资源创建在 `init_runtime()` 中完成。

init_runtime

API	**`init_runtime`**
功能	初始化运行时环境。需要在调用推理前调用此方法。
参数	`target`：指定的协处理器，目前支持 rk1820/rk1828/rk3572
	`core_mask`：NPU Core 位掩码。不同平台支持的 NPU Core 数量不同，请根据实际平台、模型转换时的 Core 配置以及 `RKNN3_QUERY_CORE_NUMBER` 查询结果设置。例如 0x0f（即二进制 0b00001111）
	`llm_args`：LLM 模型的配置参数字典
	`llm_callback`：LLM 模型的回调函数，用于流式推理
	`device_id`：设备 ID，用于区分多个设备
	`decrypt_key_path`：解密密钥或 license 文件路径，用于加载加密 RKNN 模型
	`decrypt_key_data`：解密密钥原始数据或字节流，用于加载加密 RKNN 模型；与 `decrypt_key_path` 二选一
	`postprocess_plugin_path`：后处理插件共享库（`.so`）路径。仅适用于非 LLM 模式；插件会在 `model_init` 之后、创建张量之前注册，注册后输入输出数量、输出属性和动态 Shape 查询会自动使用 `RKNN3_QUERY_POSTPROCESS_*` 命令
返回值	成功：0；失败：-1

注:

- `llm_args` 目前可配置的参数及其含义如下:
  - `top_k`：采样时考虑的最高概率词汇的数量，整数类型，默认值根据模型而定。
  - `top_p`：核采样概率阈值，浮点数类型，取值范围 [0, 1]，用于控制采样的多样性。
  - `temperature`：控制随机性的温度参数，浮点数类型，通常大于 0，越小生成越确定。

- `repeat_penalty`：重复惩罚系数，浮点数类型，用于对重复生成施加惩罚，以减少重复内容。
- `frequency_penalty`：频率惩罚，浮点数类型，基于词汇出现频率的惩罚。
- `presence_penalty`：存在惩罚，浮点数类型，基于词汇是否出现过的惩罚。
- `vocab_size`：词汇表大小，整数类型，表示模型词汇表的总大小。
- `special_bos_id`：开始标记（Beginning of Sequence）的 ID，整数类型。
- `special_eos_id`：结束标记（End of Sequence）的 ID，整数类型。
- `linefeed_id`：换行符的 ID，整数类型。
- `skip_special_token`：是否跳过特殊标记，布尔类型，True 表示跳过。
- `ignore_eos_token`：是否忽略结束标记，布尔类型，True 表示忽略。
- `keep_history`：是否保持对话历史，布尔类型，True 表示保持。
- `max_new_tokens`：最大新生成标记数，整数类型，控制生成文本的最大长度。
- `logits_name`：logits 输出的名称，字符串类型，用于指定输出层名称。
- `max_context_len`：最大上下文长度，整数类型，表示模型能处理的上下文最大长度。

- `llm_callback` 用于注册 LLM 推理过程中的回调函数，支持传入 `RKLLMCallback` 结构体或包含同名键的字典，各回调均可按需提供：

  - `result_callback`：结果回调，接收推理生成的 token 及运行状态，流式输出文本通常通过该回调实现；
  - `sampling_callback`：采样回调，需要自定义采样策略时提供；
  - `tokenizer_callback`：分词回调，需要自定义分词时提供；
  - `embed_callback`：嵌入回调，需要自定义 token 嵌入查表时提供；
  - `output_callback`：输出回调，需要获取模型输出张量时提供；
  - `input_callback`：输入回调，需要在推理过程中填充或更新指定输入张量时提供。

  每个回调均有对应的 `*_userdata` 字段，用于向回调传递用户自定义数据；未提供的回调由 Runtime 按默认逻辑处理。各回调的函数签名、参数和返回值详见 4.8 节。

- `device_id` 可以通过 `get_devices_id()` 函数获取所有协处理器的设备 ID。
- 若希望先查询模型信息再初始化 LLM 会话，可以将 `llm_args` 和 `llm_callback` 置空，查询完成后再单独调用 `init_llm_session`。
- `decrypt_key_path` 和 `decrypt_key_data` 用于加密模型加载，二者不应同时设置。
- `postprocess_plugin_path` 不支持 LLM 模式。该参数取代了原先单独暴露的 `register_custom_ops_plugins` 方法，用户应在 `init_runtime` 阶段传入插件路径。

传统模型举例如下：

```
...
# 获取device id
device_id = rknn_lite.get_devices_id()
# 初始化运行时环境
ret = rknn_lite.init_runtime(target='rk1820', core_mask=0x01,
device_id=device_id[0])
if ret != 0:
    print('Init runtime environment failed')
    exit(ret)
```

大模型举例如下：

```
ARGS = [{"max_new_tokens":256, "top_k":1, "repeat_penalty":1.1, "special_eos_id":
151645, ...}]
callback = RKLLMCallback()
...

ret = rknn_lite.init_runtime(target='rk1820', core_mask=0xff, llm_args=ARGS,
llm_callback=callback)
if ret != 0:
  print('Init runtime environment failed')
  exit(ret)
```

加密模型举例如下：

```
ret = rknn_lite.init_runtime(target='rk1820', core_mask=0x01,
decrypt_key_path='./license.key')
if ret != 0:
    print('Init encrypted model failed')
    exit(ret)
```

后处理插件举例如下：

```
ret = rknn_lite.init_runtime(
    target='rk1820',
    core_mask=0x01,
    postprocess_plugin_path='./librknn3_postprocess.so'
)
if ret != 0:
    print('Init runtime with postprocess plugin failed')
    exit(ret)
```

使用说明：

- `core_mask` 应与模型转换及运行时的 Core 配置匹配。可先查询 Core 数量，再根据有效 Core 构造对应 bit mask。
- 对 LLM，若需要先查询模型配置，可不传 `llm_args` 和 `llm_callback`；Runtime 初始化完成后通过 `rknn3_query(RKNN3_QUERY_LLM_CONFIG)` 获取配置，再调用 `init_llm_session()`。

- 使用后处理插件时，后续输出数量、输出属性和动态 Shape 查询应以注册插件后的结果为准。

3.3 通用模型推理

传统模型推理前，建议先查询输入张量属性，确认输入数量、Shape、数据类型和 Layout，再准备对应的 `numpy.ndarray` 输入。

inference

API	`inference`
功能	运行传统模型推理。
参数	`inputs`：输入数据列表，每个元素为numpy.ndarray
	`data_format`：数据格式列表，每个元素为字符串，可选 'undefined', 'nhwc' 或 'nchw'。'nhwc' 和 'nchw'仅对4维输入有效。对于非四维输入，应设为 undefined 或留空。默认值为 None。
返回值	输出数据列表，每个元素为 `numpy.ndarray`；失败时返回 `None`

示例：

```
outputs = rknn_lite.inference(inputs=[input_data], data_format=['nhwc'])
```

使用说明：

- `inputs` 的数量应与模型输入数量一致，可通过 `get_inputs_tensor_attr()` 或 `rknn3_query(RKNN3_QUERY_IN_OUT_NUM)` 预先确认。
- 可根据 `RKNN3TensorAttr.dtype`、`layout` 和 `shape` 构造输入；4 维图像输入可显式指定 `nchw` / `nhwc`，非图像或低维输入通常使用 `undefined`。

3.4 异步推理与等待

传统模型异步推理采用“提交—等待—取结果”的流程：

```
inference_async -> wait -> get_outputs
```

异步接口适合需要将 NPU 执行与 CPU 侧其他工作交叠的场景。调用 `get_outputs()` 前应先确认异步任务已经完成。

inference_async

API	`inference_async`
功能	异步运行传统模型推理。
参数	`inputs`：输入数据列表
	`data_format`：数据格式列表
返回值	成功：0；失败：-1

示例：

```
ret = rknn_lite.inference_async(inputs=[data])
```

wait

API	`wait`
功能	等待异步推理完成。
参数	无
返回值	成功：0；失败：-1

示例：

```
ret = rknn_lite.wait()
```

get_outputs

API	`get_outputs`
功能	获取异步推理的输出。
参数	无
返回值	输出数据列表

示例：

```
outputs = rknn_lite.get_outputs()
```

3.5 张量属性和动态 Shape

动态 Shape 模型可先通过 `get_dynamic_shape_infos()` 获取全部 Shape 组合，再根据实际输入 Shape 选择运行方式。高层 `inference()` 可根据输入 Shape 匹配对应组合；`set_shape()` 则适合需要显式切换 Shape 后再查询属性、执行 Profile 或固定运行配置的场景。

get_inputs_tensor_attr

API	`get_inputs_tensor_attr`
功能	获取模型输入张量的属性信息。
参数	无
返回值	输入张量属性列表；失败时返回 `None`

示例：

```
attrs = rknn_lite.get_inputs_tensor_attr()
```

使用说明：返回的属性可用于检查输入名称、逻辑 Shape、Layout、数据类型、对齐大小和所属 NPU Core。实际运行前可据此确认预处理结果是否与模型输入要求一致。

get_dynamic_shape_infos

API	`get_dynamic_shape_infos`
功能	获取传统模型支持的动态 Shape 组合信息。该接口仅适用于非 LLM 模式，静态 Shape 模型返回空列表。
参数	无
返回值	动态 Shape 信息列表；失败时返回 None

示例：

```
shape_infos = rknn_lite.get_dynamic_shape_infos()
```

使用说明：

- 返回结果用于枚举模型支持的 Shape 组合以及对应 `shape_id`。
- 对动态 Shape 输入，可将实际输入 Shape 与候选 Shape 逐一匹配；对于常见 4D 图像输入，还需要考虑 NCHW/NHWC 两种逻辑表示。
- 静态 Shape 模型返回空列表。

get_outputs_tensor_attr

API	`get_outputs_tensor_attr`
功能	获取模型输出张量的属性信息。
参数	无
返回值	输出张量属性列表；失败时返回 `None`

示例：

```
attrs = rknn_lite.get_outputs_tensor_attr()
```

使用说明：输出属性可用于确认输出数量、名称、Shape 与 dtype。异步 LLM Output Callback 场景还可据此为指定输出调用 `create_output_tensors()` 预分配内存。

set_shape

API	`set_shape`
功能	设置动态输入的模型形状。
参数	`shape_id`：形状 ID
返回值	成功：0；失败：-1

示例：

```
ret = rknn_lite.set_shape(0)
```

使用说明：

- `set_shape()` 用于显式激活某个动态 Shape 组合，传入值来自动态 Shape 信息中的 `shape_id`。
- 在对指定 Shape 执行性能分析前，可先调用该接口，再重新获取输入/输出属性并准备对应数据。
- 普通动态 Shape 推理不一定需要手动调用该接口；直接传入匹配 Shape 的输入时，可由高层推理流程完成对应选择。

3.6 张量与内存管理

本节接口面向需要显式控制 NPU 内存的高级场景。普通 `inference()` 通常不要求用户手动创建张量内存；当使用自定义输出 Callback、外部物理内存、FD 内存或用户管理的 weight/internal/KVCache 内存时，再使用这些接口。由 RKNN3Lite 创建的内存应使用对应销毁接口释放。

在显式内存场景中，`RKNN3TensorAttr`、`RKNN3TensorMemory` 和 `RKNN3Tensor` 的关系如下：

- `RKNN3TensorAttr`：描述张量的名称、Shape、数据类型、Layout、对齐大小等元信息；
- `RKNN3TensorMemory`：描述张量实际使用的内存，包括虚拟地址、物理地址、FD、大小等；
- `RKNN3Tensor`：通过 `attr` 和 `mem` 将张量属性与实际数据缓冲区关联起来。

LLM Output Callback 是这组接口的典型使用场景。应用先查询输出属性，再为 `RKNN3Tensor` 创建内存，将张量数组注册到 `RKLLMCallback.output_tensors`，推理完成后在回调中读取数据，最后通过 `destroy_mem()` 释放显式创建的内存。

create_mem

API	`create_mem`
功能	分配运行时内存，并可将该内存直接绑定到指定的 `RKNN3Tensor`。支持按张量属性分配和按字节大小分配两种用法。
参数	`attr`：`RKNN3TensorAttr` 实例，用于提供张量 Shape、dtype、对齐大小等信息（按张量属性分配时使用）；
	`tensor`：可选 `RKNN3Tensor` 实例。传入时，创建的内存会与该张量关联，适用于 Output Callback 等需要同时保存 `attr` 与 `mem` 的场景；
	`size`：分配字节数（按字节大小分配时使用）。也可传入 `numpy.ndarray`、`bytes` 等对象，此时自动取 `nbytes`；
	`core_id`：目标 NPU Core ID，默认值为 0，仅在按字节大小分配时生效；
	`flags`：内存分配标志，默认值为 `RKNN3_FLAG_MEMORY_CACHEABLE`。
返回值	成功时返回 `RKNN3TensorMemory` 实例，失败时返回 `None`。

示例：

```
# 用法一：按张量属性分配（仅创建内存）
mem = rknn_lite.create_mem(attr)

# 用法一：为输出张量创建并绑定内存
output_tensor = RKNN3Tensor()
mem = rknn_lite.create_mem(attr, output_tensor)

# 用法二：按字节大小分配（不绑定张量）
mem = rknn_lite.create_mem(size=3145728, core_id=0)

# 用法二：numpy 数组按 nbytes 分配
mem = rknn_lite.create_mem(np.zeros(1024, np.float16))
```

使用说明：

- 普通 `inference()` 通常不需要手动调用该接口；

- LLM Output Callback 场景可先通过 `RKNN3_QUERY_OUTPUT_ATTR` 获取输出属性，再调用 `create_mem(attr, tensor)` 为输出张量分配缓冲区；
- 按字节大小分配适用于不绑定模型张量的内存，例如分页 KV Cache 的单个 page，详见 3.10 节的 `set_kvcache_pages`；
- `tensor` 非空时，`attr` 必须是 `RKNN3TensorAttr` 实例，否则返回 `None`；
- 显式创建的内存不应只依赖 Python 对象回收，应在使用结束后调用 `destroy_mem()`。

兼容性说明：`flags` 位于参数列表末尾。为兼容旧版本调用方式，`create_mem(attr, tensor, flags)` 仍可正常工作；若把 `flags` 作为第 3 个位置参数传入，会被自动识别为内存分配标志，而不会当作字节大小处理。

create_mem_from_phys

API	**`create_mem_from_phys`**
功能	从物理地址创建张量内存。
参数	`phys_addr`：物理地址
	`virt_addr`：虚拟地址
	`size`：大小
返回值	`RKNN3TensorMemory` 实例，或 `None`

示例：

```
mem = rknn_lite.create_mem_from_phys(phys_addr, virt_addr, size)
```

create_mem_from_fd

API	**`create_mem_from_fd`**
功能	从文件描述符创建张量内存。
参数	`fd`：文件描述符
	`virt_addr`：虚拟地址
	`size`：大小
	`offset`：偏移
返回值	`RKNN3TensorMemory`，或 `None`

示例：

```
mem = rknn_lite.create_mem_from_fd(fd, virt_addr, size)
```

destroy_mem

API	`destroy_mem`
功能	销毁张量内存。
参数	`mem`：RKNN3TensorMemory实例
返回值	成功：0；失败：-1

示例：

```
ret = rknn_lite.destroy_mem(mem)
```

使用说明：使用 Output Callback 时，可通过 `create_output_tensors()` 创建输出内存，并在退出前遍历 `output_tensors[i].mem` 调用 `destroy_mem()`。因此由应用显式创建的输出内存应与其生命周期成对管理。

mem_sync

API	`mem_sync`
功能	同步整个张量内存，使 CPU 与设备侧数据保持一致。
参数	`mem`：RKNN3TensorMemory
	`sync_mode`：RKNN3MemSyncMode 枚举值
返回值	成功：0；失败：-1

示例：

```
ret = rknn_lite.mem_sync(mem, sync_mode)
```

mem_sync_range

API	`mem_sync_range`
功能	同步张量内存中的指定字节范围，适用于只更新部分缓冲区的场景。
参数	`mem`：RKNN3TensorMemory
	`sync_mode`：RKNN3MemSyncMode 枚举值
	`offset`：从 `mem.virt_addr` 起算的字节偏移
	`length`：需要同步的字节数，必须满足 `offset + length <= mem.size`
返回值	成功：0；失败：-1

示例：

```
ret = rknn_lite.mem_sync_range(mem, sync_mode, offset=0, length=4096)
```

set_internal_mem

API	`set_internal_mem`
功能	为多个 NPU 核心设置用户分配的内部内存。列表中每个内存对象通过其 `core_id` 指定目标核心。
参数	`mem_list`：RKNN3TensorMemory 指针列表
返回值	成功：0；失败：-1

示例：

```
ret = rknn_lite.set_internal_mem(mem_list)
```

set_weight_mem

API	`set_weight_mem`
功能	为多个 NPU 核心设置用户分配的权重内存。当前版本仅支持 RK3572。
参数	`mem_list`：RKNN3TensorMemory 指针列表；每个对象的 `core_id` 指定目标核心
返回值	成功：0；失败：-1

示例：

```
ret = rknn_lite.set_weight_mem(mem_list)
```

set_input_name_alias

API	`set_input_name_alias`
功能	建立模型实际输入名称与 Runtime/Session 期望输入名称之间的别名映射。适用于模型输入名与会话固定输入名不一致的场景。
参数	`src_input_name`：模型中实际使用的输入名称，例如 `input_embedding`
	`dst_input_name`：Runtime/Session 期望的输入名称，例如 `input_embeds`
返回值	成功：0；失败：-1

示例：

```
ret = rknn_lite.set_input_name_alias('input_embedding', 'input_embeds')
```

3.7 LLM Session 生命周期

一个 Runtime 可以管理多个 LLM Session。建议使用连续的 `session_index` 创建会话，并为每个 Session 单独设置 Chat Template、KVCache 策略和推理参数。不同 Session 可共享同一模型 Runtime，但各自维护会话状态与 KVCache。

init_llm_session

API	`init_llm_session`
功能	单独初始化 LLM 会话。在某些场景下，用户可能希望先查询模型信息再初始化 LLM 会话。此时可以单独调用 `init_llm_session`。该接口也可用于按顺序初始化额外的 LLM Session。
参数	`llm_args`：LLM 配置参数字典，或配置参数字典列表
	`llm_callback`：回调函数，可以为 dict 或 RKLLMCallback 结构
	`session_index`：LLM Session 索引，默认值为 0。初始化额外 Session 时，需按照当前 Session 数量连续指定。
返回值	成功：0；失败：-1

注：

建议的工作流如下：

1. `load_rknn(model_path, weight_path)`
2. `init_runtime(target, core_mask)`（不传入`llm_args`和`llm_callback`参数，此时仅执行 rknn3_init + 模型加载 + 模型初始化）
3. `rknn3_query(...)`（查询模型信息，如IO、核心数、内存大小等）
4. `init_llm_session(llm_args, callback)`（执行 rknn3_session_init + 设置回调）
5. `session_run(...)`（执行推理）

示例：

```
ret = rknn_lite.init_runtime(target='rk1820', core_mask=LLM_CORE_MASK)

... # 执行rknn3_query

ret = rknn_lite.init_llm_session(llm_args=ARGS, llm_callback=callback)
```

多 Session 初始化示例：

```
ret = rknn_lite.init_llm_session(llm_args=ARGS, llm_callback=callback,
session_index=1)
```

使用说明：

- 推荐在`RKNN3_QUERY_LLM_CONFIG`查询完成后，根据实际`vocab_size`、`max_ctx_len`等信息构造`llm_args`。
- 多 Session 场景使用连续的`session_index`创建会话，例如 0、1、2；每个 Session 可注册不同 Callback。
- Session 初始化完成后，通常继续配置 Chat Template 和 KVCache 策略，再调用`session_run()`。

get_session_count

API	**`get_session_count`**
功能	获取当前已初始化的 LLM Session 数量。
参数	无
返回值	成功时返回 Session 数量；失败时返回 -1

示例：

```
session_count = rknn_lite.get_session_count()
```

使用说明：用于获取当前已初始化的 LLM Session 数量，可辅助应用分配新的 `session_index` 或进行资源检查。释放单个 Session 后，应用应重新确认当前有效 Session 范围。

3.8 LLM 推理控制

LLM 推理既支持同步执行，也支持异步提交和运行中控制。同步 `session_run()` 在本次推理完成后返回；`session_run_async()` 依赖结果 Callback 判断真正结束。`session_stop()`、`session_pause()` 和 `session_resume()` 可在推理进行期间调用，并可结合 `session_query_state()` 获取当前 token、KVCache 和 LoRA 状态。

session_run

API	`session_run`
功能	运行 LLM 模型推理，支持流式输出。
参数	`inputs`：输入数据列表，用于多模态模型（如图像、音频等）。类型为列表，元素可以是 RKNN3Image、RKNN3Audio、RKNN3Video 或 RKNN3AuxTensor 等专用类型。
	`prompt`：文本提示，作为 LLM 模型的输入。
	`embeds`：输入嵌入向量，用于跳过 tokenization 的场景。类型为 numpy.ndarray，通常为二维或三维浮点张量。
	`tokens`：一维 token ID 数组（ndarray 或 list），会自动转换为 int32 类型。适用于已预先完成分词、希望直接传入原始 token ID 而非文本 prompt 的场景。
	`keep_history`：是否在推理过程中保留历史上下文，这可能会影响性能。类型为 bool，默认值为 None，表示使用模型的默认设置。
	`max_new_tokens`：推理过程中生成的最大 token 数量，这可能会影响性能。类型为 int，默认值为 None，表示使用模型的默认设置。
	`prefill_only`：是否只执行一次 Prefill。类型为 bool，默认值为 None，表示使用会话中的默认设置。
	`disable_sampling`：是否在单次 Prefill 后禁用 token 采样。类型为 bool，默认值为 None，表示使用会话中的默认设置。
	`keep_grammar`：是否在多次推理之间保留 grammar matcher 状态。类型为 bool，默认值为 None，表示使用会话中的默认设置（false）。启用后，每次 `session_run()` 前后不再重新初始化 grammar matcher，便于跨多次 run 进行连续受约束生成。
	`enable_thinking`：是否在推理过程中启用思考模式，这可能会影响性能。类型为 bool，默认值为 False，表示不使用思考模式。
	`session_index`：LLM Session 索引，默认值为 0。
返回值	`ret`（0 表示成功，-1 表示失败）以及性能统计列表 `[n_decode_tokens, n_prefill_tokens, llm_start_time, llm_end_time]`

注：`prompt`、`embeds` 和 `tokens` 三者互斥；`inputs` 用于多模态或辅助输入，可以按模型需求与 `prompt` 配合使用。

示例： LLM 模型推理代码参考如下：

```
...
prompt = "介绍一下LLM模型的工作原理。"
ret, [n_decode_tokens, n_prefill_tokens, llm_start_time, llm_end_time] =
rknn_llm.session_run(prompt=prompt)
if ret != 0:
  print('RKNN llm inference failed!')
  exit(ret)
```

VLM 模型推理代码参考如下：

```
...
# 获取视觉模型的输出并转换为所需格式
outputs = np.float16(np.expand_dims(rknn_vision.inference(inputs=[feature])[0],
0))

prompt = "<image>请描述图片内容"
inputs = []
llm_input = RKNN3Image()
llm_input.image_embed = outputs.ctypes.data_as(ctypes.POINTER(Float16))
llm_input.n_image_tokens = outputs.shape[1]
llm_input.n_image = outputs.shape[0]
llm_input.image_width = 392
llm_input.image_height = 392
llm_input.image_start = "<|vision_start|>".encode('utf-8')
llm_input.image_end = "<|vision_end|>".encode('utf-8')
llm_input.image_content = "<|image_pad|>".encode('utf-8')
inputs.append(llm_input)

# 运行LLM推理
ret, [n_decode_tokens, n_prefill_tokens, llm_start_time, llm_end_time] =
rknn_llm.session_run(inputs=inputs, prompt=prompt)
if ret != 0:
  print('RKNN LLM inference failed!')
  exit(ret)
```

使用说明：

- `prompt`、`embeds`、`tokens` 三种主输入互斥；`inputs` 可额外携带 `RKNN3Image`、`RKNN3Audio`、`RKNN3Video` 或 `RKNN3AuxTensorWrapper`。
- `keep_history`、`prefill_only`、`disable_sampling`、`keep_grammar` 等布尔参数支持传入 `0` / `1` 等整型值，内部会自动转换为 `bool` 后再传给 Runtime；`None` 表示使用会话默认设置。
- 独立请求或评测任务可使用 `keep_history=False`，避免不同输入之间共享上下文；连续对话可使用 `keep_history=True` 复用历史上下文。
- 性能测试可直接传预计算 `embeds`，避免把 Tokenizer/Embedding 查表时间计入 NPU 推理过程。
- 返回的 `n_prefill_tokens`、`n_decode_tokens`、`llm_start_time`、`llm_end_time` 可与首 token 回调时间结合计算 TTFT、Prefill TPS 和 Decode TPS。

session_run_async

API	`session_run_async`
功能	异步运行 LLM 模型推理。
参数	`inputs`：输入数据列表，用于多模态模型
	`prompt`：文本提示
	`embeds`：嵌入向量
	`tokens`：一维 token ID 数组（ndarray 或 list），会自动转换为 int32 类型。适用于已预先完成分词、希望直接传入原始 token ID 而非文本 prompt 的场景。
	`keep_history`：是否保留历史上下文
	`max_new_tokens`：最大新生成 token 数量
	`prefill_only`：是否只执行一次 Prefill，默认值为 None
	`disable_sampling`：是否在单次 Prefill 后禁用 token 采样，默认值为 None
	`keep_grammar`：是否在多次推理之间保留 grammar matcher 状态，默认值为 None（false）。启用后，每次 `session_run()` 前后不再重新初始化 grammar matcher，便于跨多次 run 进行连续受约束生成。
	`enable_thinking`：是否启用思考模式
	`session_index`：LLM Session 索引，默认值为 0。
返回值	ret, stats，stats 格式为 `[n_decode_tokens, n_prefill_tokens, llm_start_time, llm_end_time]`

注：`prompt`、`embeds` 和 `tokens` 三者互斥；`inputs` 可按模型需求与 `prompt` 配合使用。

示例：

```
ret, stats = rknn_lite.session_run_async(prompt="Async prompt")
```

使用说明：

- 接口的返回值仅表示异步任务已提交，并不表示生成已经完成；真正结束应以结果 Callback 的 FINISH/STOP/MAX_NEW_TOKEN_REACHED/ERROR 等终态为准。
- `keep_history`、`prefill_only`、`disable_sampling`、`keep_grammar` 等布尔参数支持传入 `0` / `1` 等整型值，内部会自动转换为 `bool` 后再传给 Runtime；`None` 表示使用会话默认设置。
- 异步运行期间可调用 `session_query_state()` 轮询 token 和 KVCache 状态。
- 同一 Session 连续调用异步接口更适合作为提交/排队使用；需要真正并行的多请求场景，应优先使用多个独立 Session。

session_stop

API	`session_stop`
功能	停止运行中的 LLM 会话。
参数	`session_index`：LLM Session 索引，默认值为 0。
返回值	成功：0；失败：-1

示例：

```
ret = rknn_lite.session_stop()
```

使用说明：应用可在 `LLMResultCallback` 中根据业务条件调用 `session_stop()`，随后结果回调进入 `RKLLM_RUN_STOP` 终态。该方式适合业务侧自定义停止词、外部取消请求或超时控制。

session_pause

API	`session_pause`
功能	暂停正在运行的 LLM 会话。
参数	`session_index`：LLM Session 索引，默认值为 0。
返回值	成功：0；失败：-1

示例：

```
ret = rknn_lite.session_pause()
```

使用说明：`session_pause()` 可在 Prefill 或 Decode 阶段触发暂停。暂停成功后 Session 状态会被保留，可在稍后调用 `session_resume()` 继续当前推理，而不是重新发起一次请求。

session_resume

API	`session_resume`
功能	恢复已暂停的 LLM 会话。
参数	`session_index`：LLM Session 索引，默认值为 0。
返回值	成功：0；失败：-1

示例：

```
ret = rknn_lite.session_resume()
```

使用说明：该接口应与 `session_pause()` 配对使用。结果 Callback 可收到 `RKLLM_RUN_PAUSE` 和 `RKLLM_RUN_RESUME` 状态，应用可据此更新 UI 或任务状态。

session_query_state

API	**`session_query_state`**
功能	查询 LLM 会话的当前状态。
参数	`session_index`：LLM Session 索引，默认值为 0。
返回值	成功时返回 RKLLMRunState，否则返回 None

示例：

```
state = rknn_lite.session_query_state()
```

使用说明：

- 常用字段包括 `n_total_tokens`、`n_max_tokens`、`n_prefill_tokens`、`n_decode_tokens`、`kvcache_policy` 和 `n_loras_enabled`。
- 可根据 `n_total_tokens >= n_max_tokens - max_new_tokens` 判断上下文是否接近满载，并按业务需要主动清理缓存。
- 多 Session 时应传入对应 `session_index`，状态只反映该 Session。

set_llm_param

API	**`set_llm_param`**
功能	设置已初始化会话的 LLM 参数。
参数	`params`：RKNN3LLMParam结构
	`n_params`：参数数量，默认值为 1
	`session_index`：LLM Session 索引，默认值为 0。
返回值	成功：0；失败：-1

示例：

```
ret = rknn_lite.set_llm_param(params, 1)
```

3.9 Chat Template、Function Calling 和 Grammar

Chat Template 决定用户 Prompt 在送入 Tokenizer 前如何拼接系统提示、用户前缀和助手前缀。若输入 Prompt 已由应用完整拼接，可将模板设置为空字符串，使 Prompt 原样送入模型；连续对话场景则可设置模型对应的 system/prefix/postfix。模板可以在同一 Session 的不同阶段重新设置。

Grammar（受约束生成）用于限制 LLM 的输出格式，例如固定字段 JSON、分类标签、订单号或 Tool Calling 结构。它在生成过程中限制可选 token，因此可保证输出满足指定的语法或结构；但不保证输出内容在业务或事实层面正确，应用仍应完成必要的业务校验。

set_chat_template

API	`set_chat_template`
功能	设置 LLM 的聊天模板。
参数	`system_prompt`：系统提示
	`prompt_prefix`：提示前缀
	`prompt_postfix`：提示后缀
	`session_index`：LLM Session 索引，默认值为 0。
返回值	成功：0；失败：-1

示例：

```
...
system_prompt  = "<|im_start|>system\nYou are a helpful assistant.<|im_end|>\n"
prompt_prefix  = "<|im_start|>user\n"
prompt_postfix = "<|im_end|>\n<|im_start|>assistant\n"

ret = rknn_lite.set_chat_template(system_prompt, prompt_prefix, prompt_postfix)
if ret != 0:
    print('Set chat template failed!')
    exit(ret)
```

使用说明：

- 常规对话模型通常设置 system prompt、user prefix 和 assistant postfix。
- 传入三个空字符串可使输入 Prompt 原样进入 Tokenizer，适用于应用已经自行完成 Prompt 格式拼接的场景。
- Chat Template 属于 Session 配置，可根据不同业务阶段或对话轮次动态更新。

set_function_tools

API	`set_function_tools`
功能	设置 LLM 会话的函数工具（函数调用）。
参数	`tools`：函数工具字符串（JSON 格式）
	`session_index`：LLM Session 索引，默认值为 0。
返回值	成功：0；失败：-1

示例：

```
ret = rknn_lite.set_function_tools('{"tools": []}')
```

init_grammar

API	`init_grammar`
功能	初始化当前 RKNN Context 共用的 Grammar 引擎，并建立 token ID 到 token piece 的映射。必须在 `init_runtime()` 成功后、首次调用 `set_grammar()` 前执行；同一 Context 只需初始化一次。
参数	`config`：可选的 `RKNN3GrammarConfig`。用于直接传入已构造的底层词表配置。不能与 `token_pieces` 同时传入。
	`token_pieces`：可迭代的 token piece 序列，按 token ID 顺序排列；每个元素为 `str` 或 bytes-like 对象。高层 API 会据此自动构造 `RKNN3GrammarConfig`。
	`stop_tokens`：可选的停止 token ID 列表，通常设置为 Tokenizer 的 EOS token ID。仅在通过 `token_pieces` 自动构造配置时使用。
返回值	成功：0；失败：-1。参数类型或参数组合不合法时可能抛出 Python 异常。

示例：

```
# llm_config 由 RKNN3_QUERY_LLM_CONFIG 查询得到，tokenizer 与模型词表必须匹配。
token_pieces = (
    tokenizer.decode([token_id], skip_special_tokens=False,
                     clean_up_tokenization_spaces=False)
    for token_id in range(llm_config.vocab_size)
)
stop_tokens = ([tokenizer.eos_token_id]
               if tokenizer.eos_token_id is not None else [])

ret = rknn_lite.init_grammar(
    token_pieces=token_pieces,
    stop_tokens=stop_tokens,
)
if ret != 0:
    print('Init grammar failed!')
    exit(ret)
```

使用说明：

- `token_pieces` 的第 *i* 项必须对应模型词表中的 token ID *i*；Tokenizer、Embedding 和 RKNN 模型必须来自同一模型版本。
- 生成器可作为 `token_pieces` 传入，但其内容会在初始化时被一次性读取，不可在之后重复使用。
- 若 Runtime 动态库不支持 Grammar 相关接口，调用会失败并返回 `-1`。

set_grammar

API	**`set_grammar`**
功能	为已初始化的 LLM Session 设置本次或后续请求使用的结构化输出 Grammar。调用前必须已完成 `init_grammar()` 和 `init_llm_session()`。
参数	`grammar_str`：Grammar 内容，类型为 `str`、`bytes` 或 `None`。JSON Schema、EBNF、Regex 和 Structural Tag 类型传入对应规则文本；使用 Builtin JSON 时通常传入 `None`。
	`grammar_type`：Grammar 类型，使用 `RKNN3GrammarType` 枚举。未传入时等同于 `RKNN3_GRAMMAR_BUILTIN_JSON`。
	`options`：可选的 `RKNN3GrammarSetOptions` 配置。传入 `None` 时不设置额外选项；需要使用 Runtime 默认初始化值时，可通过 `RKNN3GrammarSetOptions.default()` 创建。
	`session_index`：LLM Session 索引，默认值为 0。Grammar 配置属于指定 Session。
返回值	成功：0；失败：-1。`grammar_str` 或 `options` 类型不合法时可能抛出 Python 异常。

示例：

```
import json
from rknn3lite.api.rknn3_types import RKNN3GrammarType

person_schema = json.dumps({
    'type': 'object',
    'properties': {
        'name': {'type': 'string'},
        'age': {'type': 'integer', 'minimum': 0, 'maximum': 150},
    },
    'required': ['name', 'age'],
    'additionalProperties': False,
})

ret = rknn_lite.set_grammar(
    person_schema,
    RKNN3GrammarType.RKNN3_GRAMMAR_JSON_SCHEMA,
)
if ret != 0:
    print('Set grammar failed!')
    exit(ret)

ret, stats = rknn_lite.session_run(
    prompt='Extract name and age from: Alice is 28 years old. Reply only with
JSON.',
    keep_history=False,
    max_new_tokens=128,
)
```

Builtin JSON、Regex 和 EBNF 的使用示例：

```
# 限制为任意合法 JSON
rknn_lite.set_grammar(
    None, RKNN3GrammarType.RKNN3_GRAMMAR_BUILTIN_JSON
)

# 限制为订单号格式
rknn_lite.set_grammar(
    r'ORD-[0-9]{6}', RKNN3GrammarType.RKNN3_GRAMMAR_REGEX
)

# 限制为固定标签之一
rknn_lite.set_grammar(
    'root ::= "positive" | "negative" | "neutral"\\n',
    RKNN3GrammarType.RKNN3_GRAMMAR_EBNF,
)
```

使用说明：

- 在每次 `session_run()` 或 `session_run_async()` 前设置所需规则。不同请求需要不同规则时，应先调用 `clear_grammar()`，再设置新规则。
- 默认情况下，每次 `session_run()` 前后 grammar matcher 会重新初始化。若需跨多次 `session_run()` 保持 grammar 匹配状态（例如将一次长响应拆成多次 run 连续生成），可在

`session_run()` 传入 `keep_grammar=True`；此时不会在 run 前后重置 matcher。

- JSON Schema、Regex、EBNF 和 Structural Tag 的规则语义由 Runtime 的 Grammar 引擎解析。建议从简单规则开始验证，并为生成设置合适的 `max_new_tokens`。
- Grammar 只约束输出格式，Prompt 仍应清楚说明需要生成的内容；建议提示模型“仅输出”目标格式。

clear_grammar

API	`clear_grammar`
功能	清除指定 LLM Session 当前设置的 Grammar，使后续推理不再受该规则约束。
参数	`session_index`：LLM Session 索引，默认值为 0。
返回值	成功：0；失败：-1。

示例：

```
ret = rknn_lite.clear_grammar()
if ret != 0:
    print('Clear grammar failed!')
    exit(ret)

# Grammar 状态与 KVCache 相互独立；独立请求通常还需要清理上下文缓存。
rknn_lite.clear_kvcache()
```

使用说明： `clear_grammar()` 只清除输出约束，不清除会话历史或 KVCache。多个 Session 分别维护其 Grammar 配置，调用时应传入对应的 `session_index`。

3.10 KVCache 管理

KVCache 接口同时覆盖缓存策略、持久化、清理、用户内存和长度分组。常见使用流程为：

```
init_llm_session -> set_kvcache_policy -> session_run
                                    -> save_kvcache/load_kvcache
                                    -> session_query_state -> clear_kvcache
```

KVCache 支持通过文件路径或内存数据进行加载。应用可结合 `n_total_tokens` 与 `n_max_tokens` 判断上下文是否接近上限。

set_kvcache_policy

API	`set_kvcache_policy`
功能	设置 KVCache 策略。
参数	`policy`：RKNN3KVCachePolicy枚举值
	`param`：`RKNN3KVCachePolicyParam` 结构，或 `None`
	`session_index`：LLM Session 索引，默认值为 0。
返回值	成功：0；失败：-1

示例：

```
ret =
rknn_lite.set_kvcache_policy(RKNN3KVCachePolicy.RKNN3_KVCACHE_POLICY_DEFAULT)
```

使用说明：

- `RKNN3_KVCACHE_POLICY_NORMAL` 适用于常规上下文缓存；`RKNN3_KVCACHE_POLICY_RECURRENT` 适用于需要循环复用缓存空间的场景。具体策略应根据模型和业务需求选择。
- 多 Session 时 KVCache 策略按 `session_index` 分别设置。
- 如果策略需要额外参数，应同时传入 `RKNN3KVCachePolicyParam`。

load_kvcache

API	`load_kvcache`
功能	从文件或内存加载 KVCache。
参数	`kvcache_path`：KVCache 文件路径
	`kvcache_data`：内存中的 KVCache 数据
	`size`：数据大小
	`session_index`：LLM Session 索引，默认值为 0。
返回值	成功：0；失败：-1

示例：

```
ret = rknn_lite.load_kvcache(kvcache_path='cache.bin')
```

使用说明：支持两种来源：直接传 `kvcache_path`，或将文件读入内存后通过 `kvcache_data + size` 加载。加载成功后可继续调用 `session_run()`，用于恢复此前保存的上下文缓存状态。

save_kvcache

API	**`save_kvcache`**
功能	保存 KVCache 到文件。
参数	`kvcache_path`：保存路径
	`session_index`：LLM Session 索引，默认值为 0。
返回值	成功：0；失败：-1

示例：

```
ret = rknn_lite.save_kvcache('cache.bin')
```

使用说明：通常在完成一段 Prefill/Decode 后保存当前 Session 的 KVCache。后续可以在同模型、匹配的会话配置下通过 `load_kvcache()` 恢复，以减少重复上下文计算。

clear_kvcache

API	**`clear_kvcache`**
功能	清除 KVCache。
参数	`clear_policy`：`RKNN3KVCacheClearPolicy` 枚举值，或 `None`
	`param`：`RKNN3KVCacheClearPolicyParam` 结构体；当 `clear_policy` 为 `RKNN3_KVCACHE_KEEP_N_TOKENS` 时必须传入，否则可为 `None`
	`session_index`：LLM Session 索引，默认值为 0。
返回值	成功：0；失败：-1

示例：

```
ret = rknn_lite.clear_kvcache()

clear_param = RKNN3KVCacheClearPolicyParam()
clear_param.keep_n_tokens.n_tokens = 128
ret = rknn_lite.clear_kvcache(
    RKNN3KVCacheClearPolicy.RKNN3_KVCACHE_KEEP_N_TOKENS,
    param=clear_param,
)
```

使用说明：

- `RKNN3_KVCACHE_CLEAR_ALL` 用于彻底清理当前 Session 上下文，适合相互独立的评测样本。
- `RKNN3_KVCACHE_KEEP_SYSTEM_PROMPT` 可在清理 KVCache 时保留系统提示对应缓存，减少后续请求重复计算系统提示的开销。
- `RKNN3_KVCACHE_KEEP_N_TOKENS` 可在清理 KVCache 时保留前 `n_tokens` 个 token，调用时必须提供 `RKNN3KVCacheClearPolicyParam`。
- 清理操作应指定正确的 `session_index`，避免影响其他 Session。

set_kvcache_mem

API	`set_kvcache_mem`
功能	为指定 NPU 核心设置 KVCache 内存。
参数	`mem_list`：RKNN3TensorMemory列表
	`npu_core_indices`：核心索引列表
返回值	成功：0；失败：-1

示例：

```
ret = rknn_lite.set_kvcache_mem(mem_list, [0,1])
```

set_kvcache_group

API	`set_kvcache_group`
功能	设置当前 Context 使用的 KV Cache 长度分组。运行时会切换对应的 KV Cache 缓冲区地址和执行配置。
参数	`group_id`：需要激活的分组 ID，必须满足 `0 <= group_id < n_groups`
返回值	成功：0；失败：-1

示例：

```
infos = rknn_lite.rknn3_query(
    RKNN3QueryCmd.RKNN3_QUERY_KVCACHE_LEN_GROUP_INFO
)
if infos and infos[0].n_groups > 1:
    ret = rknn_lite.set_kvcache_group(1)
```

set_kvcache_pages

API	`set_kvcache_pages`
功能	替换指定的分页 KV Cache 绑定关系，不复制 page 内容。绑定成功后，page 由调用方持有，必须保持有效直到解绑。
参数	`bindings`：`RKNN3KvCachePageBinding` 结构列表，每个元素描述一个 (Core, 缓存类别, request 索引) 的 page 绑定。
返回值	成功：0；失败：-1

示例：

```
from rknn3lite.api.rknn3_types import (
    RKNN3QueryCmd,
    RKNN3PageCacheType,
    RKNN3MemAllocFlags,
    RKNN3MemSyncMode,
    RKNN3KvCachePageBinding,
    RKNN3TensorMemory,
)
from ctypes import POINTER

# 1) 枚举 Core，并确认模型启用了分页 KV Cache
alloc_infos = rknn_lite.rknn3_query(RKNN3QueryCmd.RKNN3_QUERY_ALLOCATION_INFO)
if not alloc_infos:
    raise RuntimeError('failed to query allocation info')
page_info = rknn_lite.rknn3_query(
    RKNN3QueryCmd.RKNN3_QUERY_KVCACHE_PAGE_INFO, index=alloc_infos[0].core_id
)
if page_info is None or page_info.n_cache_types == 0:
    raise RuntimeError('model does not use paged KV cache')

# 2) 查询注意力配置，取得 tokens_per_page 等信息
attention_configs =
rknn_lite.rknn3_query(RKNN3QueryCmd.RKNN3_QUERY_ATTENTION_CONFIG)

# 3) 按 page_size 逐个分配 page，并绑定
bindings = []
all_page_mems = []        # 统一持有全部 page，释放阶段遍历该列表
page_arrays = []          # 保持引用，避免 page 指针数组被回收
for allocation in alloc_infos:      # 每个 Core 都要绑定自己的 page 表
    info = rknn_lite.rknn3_query(
        RKNN3QueryCmd.RKNN3_QUERY_KVCACHE_PAGE_INFO, index=allocation.core_id
    )
    if info is None or info.n_cache_types == 0:
        raise RuntimeError('inconsistent paged KV cache info across cores')
    for request in range(info.max_request_num):
        for i in range(info.n_cache_types):
            cache = info.caches[i]
            page_mems = []
            for _ in range(cache.max_page_num):
                mem = rknn_lite.create_mem(
                    size=cache.page_size, core_id=info.core_id,
                    flags=RKNN3MemAllocFlags.RKNN3_FLAG_MEMORY_CACHEABLE,
                )
```

```
                rknn_lite.mem_sync(mem,
RKNN3MemSyncMode.RKNN3_MEMORY_SYNC_TO_DEVICE)
                page_mems.append(mem)
            all_page_mems.extend(page_mems)

            page_array = (POINTER(RKNN3TensorMemory) * len(page_mems))(*page_mems)
            page_arrays.append(page_array)

            binding = RKNN3KvCachePageBinding()
            binding.core_id = info.core_id
            binding.cache_type = cache.cache_type
            binding.request_index = request
            binding.n_pages = len(page_mems)
            binding.pages = page_array
            bindings.append(binding)

ret = rknn_lite.set_kvcache_pages(bindings)

# 4) 使用结束后解绑（n_pages = 0），再释放 page
unbinds = []
for binding in bindings:
    unbound = RKNN3KvCachePageBinding()
    unbound.core_id = binding.core_id
    unbound.cache_type = binding.cache_type
    unbound.request_index = binding.request_index
    unbound.n_pages = 0
    unbound.pages = None
    unbinds.append(unbound)
rknn_lite.set_kvcache_pages(unbinds)

for mem in all_page_mems:
    rknn_lite.destroy_mem(mem)
```

使用说明：

- 该接口为 Context 级操作，不属于某个 Session；绑定的 page 由当前 Context 下所有 Session 共享，只能在 Context 空闲时调用。
- 仅在分页 KV Cache 模型上使用。非分页模型 `RKNN3_QUERY_KVCACHE_PAGE_INFO` 返回的 `n_cache_types` 为 0，此时应改用 3.10 节的 `set_kvcache_mem`。
- 示例先探测 `alloc_infos[0]` 判断是否启用分页，再遍历 `alloc_infos` 为每个 Core 分别绑定 page 表；多核模型不能只绑定第一个 Core。
- `n_pages = 0` 表示解绑该条目；未出现在 `bindings` 中的条目保持原有绑定不变。
- 传入的 `bindings` 会先做完整形状校验；校验失败时保留原有绑定，不会产生部分生效。
- 若 page 表写入失败，本批次中的所有条目都会被解绑；两种情况调用方都可以释放本批次已分配的 page。
- 调用方持有成功绑定的 page，必须保持其有效直到解绑。解绑顺序应为：先用 `n_pages = 0` 解绑，再调用 `destroy_mem()` 释放 page。
- 分页模型应在绑定 page 表之前完成 internal/weight 内存设置，因为这些接口可能重写命令内存。
- 分页 KV Cache 模型不使用缓存长度分组，因此不需要调用 `set_kvcache_group()`。

3.11 LoRA 管理

LoRA 的典型生命周期为：

```
init_lora -> query_lora -> load_lora -> enable_lora(session)
                                     -> disable_lora(session) -> unload_lora
```

同一个 Base Runtime 可以创建多个 Session，并仅在指定 Session 上启用 LoRA，从而让不同 Session 分别运行 Base 模型或 LoRA 模型。

init_lora

API	`init_lora`
功能	初始化 LoRA 支持，加载 LoRA 权重数据。
参数	`lora_weight_path`：LoRA 权重文件路径或目录
	`weight_data`：可选的内存 LoRA 权重数据，支持 bytes/bytearray/memoryview、numpy.ndarray 或裸指针
	`weight_size`：`weight_data`为裸指针时的数据大小，单位为字节
	`session_index`：LLM Session 索引，默认值为 0。
返回值	成功：0；失败：-1

示例：

```
ret = rknn_lite.init_lora(lora_weight_path='lora_weights.bin')
```

使用说明：LoRA 权重既可从文件路径初始化，也可先读取成 bytes 后通过 `weight_data` / `weight_size` 从内存初始化。初始化后可先调用 `query_lora()` 获取实际适配器名称和默认 scale。

`init_lora()` 可重复调用以注册多个 LoRA 权重文件或内存缓冲；所有已注册文件/缓冲中的适配器名称必须全局唯一。通过 `weight_data` 注册的内存缓冲为非持有引用，必须在 RKNN3 Context 释放前保持有效。

load_lora

API	`load_lora`
功能	加载 LoRA 适配器。
参数	`lora`：RKNN3Lora结构，包含lora_name和scale
	`session_index`：LLM Session 索引，默认值为 0。
返回值	成功：0；失败：-1

示例：

```
lora = RKNN3Lora()
lora.lora_name = b'adapter1'
lora.scale = 1.0
ret = rknn_lite.load_lora(lora)
```

使用说明： `load_lora()` 将已经初始化的适配器加载到 Runtime 中，但不会自动使其对某个 Session 生效；需要继续调用 `enable_lora()` 指定目标 Session。

unload_lora

API	`unload_lora`
功能	卸载 LoRA 适配器。
参数	`lora`：RKNN3Lora结构，包含lora_name和scale
	`session_index`：LLM Session 索引，默认值为 0。
返回值	成功：0；失败：-1

示例：

```
ret = rknn_lite.unload_lora(lora)
```

enable_lora

API	`enable_lora`
功能	启用 LoRA 适配器用于推理。
参数	`lora`：RKNN3Lora结构，包含lora_name和scale
	`session_index`：LLM Session 索引，默认值为 0。
返回值	成功：0；失败：-1

示例：

```
ret = rknn_lite.enable_lora(lora)
```

使用说明：LoRA 可按 Session 启用。不同 Session 可以共享同一 Runtime，并分别选择是否启用 LoRA。`RKNN3Lora.scale` 可在启用前调整，用于控制适配器作用强度。

注意：映射到同一个 pattern slot 的不同适配器不能在同一个 Session 中同时启用；不同 Session 可以复用该 slot，Runtime 会在每次推理前重新应用各 Session 的 LoRA 绑定。

disable_lora

API	`disable_lora`
功能	禁用 LoRA 适配器。
参数	`lora`：RKNN3Lora结构，包含lora_name和scale
	`session_index`：LLM Session 索引，默认值为 0。
返回值	成功：0；失败：-1

示例：

```
ret = rknn_lite.disable_lora(lora)
```

使用说明：只停止 LoRA 在指定 Session 上生效，不等同于从 Runtime 卸载权重。需要彻底释放适配器时，再调用 `unload_lora()`。

query_lora

API	`query_lora`
功能	查询当前 Runtime 中可用的 LoRA 适配器信息（即 `init_lora()` 初始化得到的 LoRA 信息）。
参数	`session_index`：LLM Session 索引，默认值为 0。
返回值	(lora_list, n_lora) 成功时，否则(None, 0)

示例：

```
lora_list, n_lora = rknn_lite.query_lora()
```

使用说明：返回的 `lora_list` 中每个元素包含 `lora_name` 与 `scale`。应用可先查询实际列表，再选择其中某个适配器执行 `load_lora()` / `enable_lora()`，避免硬编码适配器名称。

3.12 Callback 配置

RKNN3Lite 没有单独的 `set_callback()` 方法。LLM Callback 通过 `init_runtime(..., llm_callback=...)` 或 `init_llm_session(..., llm_callback=...)` 注册，所有回调统一保存在 `RKLLMCallback` 中。

常见回调包括：

- `result_callback`：接收生成 token 和运行状态；
- `tokenizer_callback`：将输入文本转换为 token ID；
- `embed_callback`：根据 token ID 填充 token embedding；
- `input_callback`：在推理过程中按当前 token、位置等信息填充指定的扩展输入张量；
- `output_callback`：在 Prefill/Decode 指定阶段读取用户注册的模型输出张量。

常规文本 LLM 通常只需要结果、Tokenizer 和 Embedding 回调。对于存在额外模型输入的场景，可通过 `input_tensors_index` 和 `n_input_tensors` 指定由 `input_callback` 负责填充的输入索引；对于需要获取中间层或输出 embedding 的场景，可通过 `output_tensors` 和 `n_output_tensors` 注册 Output Callback 使用的张量。

Callback 由底层 Runtime 在推理期间调用，因此相关的 `ctypes` 回调对象、userdata、输入索引数组以及 Callback 中引用的 NumPy/内存对象都必须在 Session 使用期间保持有效，不能在初始化后立即被 Python 垃圾回收。

Callback 配置示例

下面示例展示 Output Callback 的通用配置关系：

```
io_num = rknn.rknn3_query(RKNN3QueryCmd.RKNN3_QUERY_IN_OUT_NUM)

OutputTensorArray = RKNN3Tensor * io_num.n_output
output_tensors = OutputTensorArray()

for i in range(io_num.n_output):
    attr = rknn.rknn3_query(RKNN3QueryCmd.RKNN3_QUERY_OUTPUT_ATTR, index=i)
    if rknn.create_mem(attr, output_tensors[i]) is None:
        raise RuntimeError(f"create output memory failed: {i}")

callback.output_callback = LLMOutputCallback(output_callback)
callback.output_tensors = ctypes.cast(output_tensors, ctypes.POINTER(RKNN3Tensor))
callback.n_output_tensors = io_num.n_output
```

Session 使用结束后，需要释放上述显式创建的输出内存：

```
for tensor in output_tensors:
    rknn.destroy_mem(tensor.mem)
```

对于 Input Callback，应先通过模型输入属性确定需要由应用填充的输入索引，再设置 `input_tensors_index` 和 `n_input_tensors`。具体输入名称和数据组织方式由模型决定。

create_output_tensors

API	**`create_output_tensors`**
功能	创建带分配内存的输出张量，用于 LLM 输出回调。
参数	`output_indices`：输出张量索引列表；如果为 None，则默认使用全部输出张量。
返回值	(output_tensor_array, n_output_tensors)；失败时返回 (None, 0)

示例：

```
tensors, n = rknn_lite.create_output_tensors([0,1])
```

使用说明：

- 主要用于 `LLMOutputCallback`。应用也可以自行查询 `RKNN3_QUERY_OUTPUT_ATTR`，构造 `RKNN3Tensor` 数组并逐个调用 `create_mem(attr, tensor)`；
- 创建完成后，需要将张量数组设置到 `RKLLMCallback.output_tensors`，并同步设置 `n_output_tensors`；

- 回调触发后可通过 `tensor.attr` 获取名称、Shape、dtype 等信息，通过 `tensor.mem.virt_addr` 访问实际输出数据；
- 这些张量包含显式分配的内存，Session 使用结束后应逐个调用 `destroy_mem(output_tensors[i].mem)`。

3.13 Profile 和调试

进行 Profile 时，建议先完成模型初始化和一次有效推理，再调用 `profile_mem()` 和 `profile_ops()`。动态 Shape 模型如果需要分析指定 Shape，应先切换或确认当前 Shape，避免 Profile 结果与预期输入配置不一致。

dump_features

API	**`dump_features`**
功能	将模型层间特征转储到.npy文件。
参数	`dump_dir`：转储目录
返回值	成功：0；失败：-1

示例：

```
ret = rknn_lite.dump_features('./dump')
```

profile_ops

API	**`profile_ops`**
功能	打印层间操作性能信息。
参数	`log_level`：日志级别
返回值	成功：0；失败：-1

示例：

```
ret = rknn_lite.profile_ops(1)
```

使用说明：建议先完成一次有效 `inference()`，再调用该接口，使 Profile 对应实际执行过的模型和 Shape。`log_level` 可用于控制 Profile 输出粒度，具体支持值以接口定义为准。

profile_mem

API	`profile_mem`
功能	打印每个 NPU 核心的内存使用信息。
参数	无
返回值	成功：0；失败：-1

示例：

```
ret = rknn_lite.profile_mem()
```

使用说明：推理完成后可先调用 `profile_mem()` 查看各 NPU Core 的内存占用，再调用 `profile_ops()` 分析算子性能，二者可组合用于定位性能瓶颈和内存压力。

3.14 图像内存与图像处理

本节用于 Runtime 图像内存和颜色转换。其接口与 VLM 中直接传递 `RKNN3Image.image_embed` 的方式不同：前者处理图像缓冲区，后者描述已经由视觉模型提取完成的多模态 embedding。

im_mem_create

API	`im_mem_create`
功能	创建图像内存。
参数	`width`：宽度
	`height`：高度
	`fmt`：图像格式
	`size`：大小
	`core_id`：核心 ID
	`flags`：标志
返回值	(ret, RKNN3ImMem)

示例：

```
ret, im_mem = rknn_lite.im_mem_create(224, 224, fmt, 0, 0)
```

im_mem_destroy

API	`im_mem_destroy`
功能	销毁图像内存。
参数	`im_mem`：RKNN3ImMem对象
返回值	成功：0；失败：-1

示例：

```
ret = rknn_lite.im_mem_destroy(im_mem)
```

im_cvt_color

API	`im_cvt_color`
功能	转换图像颜色空间。
参数	`src_im_mem`：源图像内存
	`dst_im_mem`：目标图像内存
返回值	成功：0；失败：-1

示例：

```
ret = rknn_lite.im_cvt_color(src, dst)
```

3.15 设备查询和上下文复制

多设备环境下可在初始化前通过 `get_devices_id()` 选择设备，并在 Runtime 初始化后使用 `rknn3_query()` 查询 Core 数量、设备内存、I/O 和张量属性。LLM 场景还应在创建 Session 前查询 `RKNN3_QUERY_LLM_CONFIG`，用于根据模型实际配置构造会话参数。

get_sdk_version

API	`get_sdk_version`
功能	获取RKNN SDK版本信息。
参数	无
返回值	SDK 版本字符串；失败时返回 `None`

示例：

```
version = rknn_lite.get_sdk_version()
print(version)
```

get_devices_id

API	`get_devices_id`
功能	获取指定 target 下所有已连接设备的 ID 列表。
参数	`target`：指定的协处理器类型，默认值为 `rk1820`。
返回值	设备 ID 列表

示例：

```
devices = rknn_lite.get_devices_id()
print(devices)
```

注：以上接口适用于多设备部署场景。若仅使用单设备，可直接取列表首元素或传入 None（如 init_runtime 所支持）。

使用说明：多设备环境下可在 `init_runtime()` 前调用该接口，并把选中的设备 ID 传给 `device_id`。如果业务没有指定设备，可从返回列表中选择一个可用设备。

dup_context

API	`dup_context`
功能	复制当前RKNN3上下文。
参数	无
返回值	(ret, new_context)

示例：

```
ret, new_ctx = rknn_lite.dup_context()
```

rknn3_query

API	`rknn3_query`
功能	通用查询接口，用于在运行时初始化后查询模型、设备、性能、动态 Shape、LLM 配置、LoRA 和后处理等信息。
参数	`cmd`：RKNN3QueryCmd枚举值
	`index`：索引值。用于输入/输出张量索引、Core ID 或 Shape ID 等场景，默认值为 0。
返回值	查询结果。不同 `cmd` 对应的返回类型如下：
	`RKNN3_QUERY_IN_OUT_NUM`： `RKNN3InputOutputNum`
	`RKNN3_QUERY_INPUT_ATTR`： `RKNN3TensorAttr`
	`RKNN3_QUERY_OUTPUT_ATTR`： `RKNN3TensorAttr`
	`RKNN3_QUERY_NATIVE_INPUT_ATTR` / `RKNN3_QUERY_NATIVE_OUTPUT_ATTR`： `RKNN3TensorAttr`
	`RKNN3_QUERY_NATIVE_NHWC_INPUT_ATTR` / `RKNN3_QUERY_NATIVE_NHWC_OUTPUT_ATTR`： `RKNN3TensorAttr`
	`RKNN3_QUERY_SDK_VERSION`： `RKNN3SDKVersion`
	`RKNN3_QUERY_CORE_NUMBER`：核心数量（ `int` ）
	`RKNN3_QUERY_CORE_MEM_SIZE`： `RKNN3CoreMemSize`
	`RKNN3_QUERY_DEVICE_MEM_INFO`： `RKNN3DevMemInfo`
	`RKNN3_QUERY_CUSTOM_STRING`： `RKNN3CustomString`
	`RKNN3_QUERY_PERF_DETAIL`： `RKNN3PerfDetail`
	`RKNN3_QUERY_PERF_RUN`： `RKNN3PerfRun`
	`RKNN3_QUERY_DYNAMIC_SHAPE_CONFIG`： `RKNN3ShapeConfig`
	`RKNN3_QUERY_DYNAMIC_SHAPE_INFO`： `RKNN3ShapeInfo`
	`RKNN3_QUERY_LLM_CONFIG`： `RKNN3LLMConfig`
	`RKNN3_QUERY_POSTPROCESS_IN_OUT_NUM`： `RKNN3InputOutputNum`
	`RKNN3_QUERY_POSTPROCESS_OUTPUT_ATTR`： `RKNN3TensorAttr`
	`RKNN3_QUERY_POSTPROCESS_DYNAMIC_SHAPE_INFO`： `RKNN3ShapeInfo`
	`RKNN3_QUERY_LORA_NUM`：LoRA 数量（ `int` ）
	`RKNN3_QUERY_LORA_INFO`： `RKNN3Lora` 列表
	`RKNN3_QUERY_KVCACHE_LEN_GROUP_INFO`： `RKNN3KvCacheLenGroupInfo` 列表（按核数分配，每个元素对应一个 NPU 核心）

API	`rknn3_query`
	`RKNN3_QUERY_ALLOCATION_INFO`：`RKNN3AllocationInfo` 列表（按核数分配，每个元素对应一个 NPU 核心）
	`RKNN3_QUERY_KVCACHE_PAGE_INFO`：`RKNN3KvCachePageInfo`（通过 `index` 指定 Core ID）
	`RKNN3_QUERY_ATTENTION_CONFIG`：`RKNN3AttentionConfig` 列表（数量取自 `RKNN3_QUERY_LLM_CONFIG` 的 `n_attention_configs`）

示例：

```
version = rknn_lite.rknn3_query(RKNN3QueryCmd.RKNN3_QUERY_SDK_VERSION)
io_num = rknn_lite.rknn3_query(RKNN3QueryCmd.RKNN3_QUERY_IN_OUT_NUM)
input_attr = rknn_lite.rknn3_query(RKNN3QueryCmd.RKNN3_QUERY_INPUT_ATTR, index=0)
```

使用说明：

- 通用模型常查询 `IN_OUT_NUM`、`INPUT_ATTR`、`OUTPUT_ATTR`、`DEVICE_MEM_INFO` 和 `CORE_NUMBER`。
- LLM 通常在 Session 初始化前额外查询 `LLM_CONFIG`，用实际 `vocab_size`、`embedding_dim`、`max_ctx_len`、`max_position_embeddings`、`task_type` 等配置构造应用参数。
- 动态 Shape、LoRA 和 KVCache 分组也通过对应 Query Cmd 获取元信息。
- 分页 KV Cache 场景需要先通过 `RKNN3_QUERY_ALLOCATION_INFO` 枚举各 Core，再逐 Core 调用 `RKNN3_QUERY_KVCACHE_PAGE_INFO`（通过 `index` 传入 `core_id`）获取 page 几何信息，并通过 `RKNN3_QUERY_ATTENTION_CONFIG` 取得 `tokens_per_page` 等配置。完整流程见 3.10 节的 `set_kvcache_pages`。

4. RKNN3 类型定义

本章节详细说明 RKNN3 Toolkit Lite 中使用的类型定义、结构、枚举和常量。这些定义基于 `rknn3_types.py` 文件，用于描述 RKNN3 API 的数据结构和参数。

4.1 常量、状态码和枚举

本节说明 RKNN3 Toolkit Lite 使用的公共常量、RKNN3 Runtime 状态码以及各类枚举定义。这些定义主要用于错误处理、Query、张量解释、内存同步、NPU Core、KVCache 和 LLM 运行状态。常见使用方式是通过 `RKNN3QueryCmd` 查询模型元信息，并结合 `RKNN3TensorType`、`RKNN3TensorLayout` 等类型解释张量属性，再据此准备输入和运行参数。

4.1.1 常量定义

RKNN3 Toolkit Lite 定义了以下常量：

名称	说明
`RKNN3_MAX_DIMS`	`16`：张量最大维度数
`RKNN3_MAX_STRIDE_DIMS`	`RKNN3_MAX_DIMS + 1`：张量步长最大维度数
`RKNN3_MAX_NAME_LEN`	`256`：名称最大长度
`RKNN3_MAX_DYNAMIC_SHAPE_NUM`	`512`：动态形状最大数量
`RKNN3_MAX_LORA_NUM`	`32`：LoRA 最大数量
`RKNN3_MAX_DEVS`	`64`：最大设备数
`RKNN3_MAX_DEV_LEN`	`64`：设备 ID 最大长度
`RKNN3_MAX_NPU_NODE_NUM`	`128`：NPU 节点最大数量
`RKNN3_MAX_KVCACHE_LEN_GROUPS`	`16`：单个注意力类型或 KV Cache 分组信息支持的最大长度分组数

名称	说明
`RKNN3_MAX_ATTENTION_TYPE_NUM`	`3`：LLM 配置支持的最大注意力类型数量
`RKNN3_MAX_CACHE_PAGE_TYPES`	`16`：分页 KV Cache 支持的最大缓存类型数量
`RKNN3_MAX_MROPE_SECTION_LEN`	`8`：mRoPE section 最大元素数量
`RKNN3_MAX_ATTENTION_CONFIG_NUM`	`32`：注意力配置最大数量
`RKNN3_MAX_SPECIAL_BOS_ID_NUM`	`64`：特殊开始标记 ID 最大数量
`RKNN3_MAX_SPECIAL_EOS_ID_NUM`	`64`：特殊结束标记 ID 最大数量
`LIBRKNN3RT_PATHS`	RKNN3 运行时库的默认候选路径列表。Linux 为 `['/usr/lib/librknn3_api.so', '/usr/lib/librknn3_api_rkcp.so', '/usr/lib/librknn3_api_native.so']`；macOS 为 `/usr/local/lib/` 下对应的 `.dylib` 文件。
`rknn3_runtime_api_ret_code`	RKNN3 Runtime 返回码到状态名称的映射。完整状态码及含义见 4.1.2 状态码。

运行时库的查找优先级如下：

1. `LD_LIBRARY_PATH` 环境变量中指定的目录；
2. `rknn3lite/api/rknn3_types.py` 所在的包目录；
3. `LIBRKNN3RT_PATHS` 中的系统默认路径。

如运行时库位于自定义目录，可在启动 Python 程序前设置 `LD_LIBRARY_PATH`：

```
export LD_LIBRARY_PATH=/path/to/rknn3/lib:${LD_LIBRARY_PATH}
```

该目录中可包含 `librknn3_api.so`、`librknn3_api_rkcp.so` 或 `librknn3_api_native.so`；程序会按上述顺序查找可用的库文件。

4.1.2 状态码

RKNN3 Runtime 使用整数状态码表示接口执行结果。`0` 表示执行成功，负值表示错误或警告。

RKNN3 Toolkit Lite 的高层 Python 接口通常会根据底层 Runtime 返回值记录错误日志，并按各接口自身约定返回 `0`、`-1`、`None` 或其他结果。因此，下表主要用于解释日志中出现的底层 RKNN3 Runtime 状态码；具体 Python 接口的返回形式仍以第 3 章对应接口说明为准。

状态码	值	描述
`RKNN3_SUCCESS`	0	执行成功
`RKNN3_ERR_FAIL`	-1	执行失败
`RKNN3_ERR_ARGUMENT_INVALID`	-2	参数无效
`RKNN3_ERR_MODEL_INVALID`	-3	模型无效
`RKNN3_ERR_CTX_INVALID`	-4	Context 无效
`RKNN3_ERR_RUN_TASK_FAILED`	-5	任务运行失败
`RKNN3_ERR_OUT_OF_MEMORY`	-6	内存不足
`RKNN3_ERR_TIMEOUT`	-7	执行超时
`RKNN3_ERR_INPUT_INVALID`	-8	输入无效
`RKNN3_ERR_OUTPUT_INVALID`	-9	输出无效
`RKNN3_ERR_DEVICE_UNAVAILABLE`	-10	设备不可用
`RKNN3_ERR_DEVICE_UNMATCH`	-11	设备不匹配
`RKNN3_ERR_TARGET_PLATFORM_UNMATCH`	-12	目标平台不匹配
`RKNN3_ERR_COMMUNICATION`	-13	通信错误
`RKNN3_ERR_MEM_SYNC_FAILED`	-14	内存同步失败
`RKNN3_ERR_KEY_INVALID`	-15	解密密钥无效或 RSA 解密失败
`RKNN3_ERR_DECRYPT_FAILED`	-16	模型解密失败，可能由密钥不匹配或数据损坏导致
`RKNN3_ERR_WEIGHT_LOAD_NOT_PREPARED`	-17	流式权重加载未准备好，底层 Runtime 需要先完成模型初始化
`RKNN3_ERR_INCOMPATIBLE_VERSION`	-18	组件版本不兼容；相关 Runtime、服务或固件版本需要保持一致
`RKNN3_ERR_MATMUL_INVALID_STATE`	-19	MatMul 任务当前状态不允许执行该操作
`RKNN3_ERR_MATMUL_COMPLETION_UNKNOWN`	-20	MatMul 任务完成状态未知，相关内存可能仍被访问；应保留相关资源，待销毁成功后再释放

状态码	值	描述
`RKNN3_WARN_NPU_CORE_UNUSED`	-100	NPU Core 未使用，仅作为警告，不影响模型执行

使用说明：

- 一般情况下，状态码为 `0` 表示底层 Runtime 操作成功；
- `-1` ～ `-20` 为错误状态，应结合接口调用位置和日志定位具体原因；
- `RKNN3_WARN_NPU_CORE_UNUSED` 的值虽然为负数，但属于警告状态，不表示模型执行失败；
- Python 高层接口可能将不同的底层错误统一转换为 `-1` 或 `None` 返回，因此需要结合终端日志或日志文件中的 Runtime 状态码进行进一步定位；
- `RKNN3_ERR_INCOMPATIBLE_VERSION` 通常表示 RKNN3 Runtime 相关组件版本不一致，应检查 Python Wheel、`librknn3_api.so`、设备侧服务以及协处理器固件之间的版本匹配关系。

4.1.3 RKNN3QueryCmd

查询命令类型枚举：

名称	值	说明
`RKNN3_QUERY_IN_OUT_NUM`	`0`	查询输入输出数量
`RKNN3_QUERY_INPUT_ATTR`	`1`	查询输入属性
`RKNN3_QUERY_OUTPUT_ATTR`	`2`	查询输出属性
`RKNN3_QUERY_PERF_DETAIL`	`3`	查询性能详情
`RKNN3_QUERY_PERF_RUN`	`4`	查询运行性能
`RKNN3_QUERY_SDK_VERSION`	`5`	查询SDK版本
`RKNN3_QUERY_CORE_MEM_SIZE`	`6`	查询核心内存大小
`RKNN3_QUERY_CUSTOM_STRING`	`7`	查询自定义字符串
`RKNN3_QUERY_ORIGIN_INPUT_ATTR`	`8`	查询原始输入属性
`RKNN3_QUERY_ORIGIN_OUTPUT_ATTR`	`9`	查询原始输出属性
`RKNN3_QUERY_NATIVE_INPUT_ATTR`	-	查询原生输入属性，兼容旧版本命名
`RKNN3_QUERY_NATIVE_OUTPUT_ATTR`	-	查询原生输出属性，兼容旧版本命名
`RKNN3_QUERY_NATIVE_NC1HWC2_INPUT_ATTR`	-	查询 NC1HWC2 输入属性，兼容旧版本命名
`RKNN3_QUERY_NATIVE_NC1HWC2_OUTPUT_ATTR`	-	查询 NC1HWC2 输出属性，兼容旧版本命名

名称	值	说明
RKNN3_QUERY_NATIVE_NHWC_INPUT_ATTR	10	查询NHWC输入属性
RKNN3_QUERY_NATIVE_NHWC_OUTPUT_ATTR	11	查询NHWC输出属性
RKNN3_QUERY_DEVICE_MEM_INFO	12	查询设备内存信息
RKNN3_QUERY_CORE_NUMBER	13	查询核心数量
RKNN3_QUERY_ALLOCATION_INFO	14	查询分配信息
RKNN3_QUERY_DYNAMIC_SHAPE_CONFIG	15	查询动态形状配置
RKNN3_QUERY_DYNAMIC_SHAPE_INFO	16	查询动态形状信息
RKNN3_QUERY_LLM_CONFIG	17	查询 LLM 配置
RKNN3_QUERY_POSTPROCESS_IN_OUT_NUM	18	查询后处理输入输出数量
RKNN3_QUERY_POSTPROCESS_OUTPUT_ATTR	19	查询后处理输出属性
RKNN3_QUERY_POSTPROCESS_DYNAMIC_SHAPE_INFO	20	查询后处理动态形状信息
RKNN3_QUERY_LORA_NUM	21	查询 LoRA 数量
RKNN3_QUERY_LORA_INFO	22	查询 LoRA 信息
RKNN3_QUERY_KVCACHE_LEN_GROUP_INFO	23	查询 KV Cache 长度分组信息
RKNN3_QUERY_KVCACHE_PAGE_INFO	24	查询指定 Core 的分页 KV Cache 信息
RKNN3_QUERY_ATTENTION_CONFIG	25	查询全部注意力配置
RKNN3_QUERY_CMD_MAX	26	查询命令最大值

注：`RKNN3_QUERY_PERF_DETAIL` 和 `RKNN3_QUERY_PERF_RUN` 已不在当前公开 C 头文件中，Python 层仅为兼容旧版 SO 保留。

使用说明：

- `RKNN3_QUERY_IN_OUT_NUM` 通常是初始化后第一项查询，用于决定后续需要读取多少个输入/输出属性。
- `RKNN3_QUERY_INPUT_ATTR` / `OUTPUT_ATTR` 与 `index` 配合使用。
- `RKNN3_QUERY_LLM_CONFIG` 可用于获取 `vocab_size`、上下文长度和模型类型等配置，避免应用侧硬编码。
- `RKNN3_QUERY_DEVICE_MEM_INFO`、`CORE_NUMBER` 可用于设备资源检查和生成合适的 `core_mask`。
- `RKNN3_QUERY_KVCACHE_PAGE_INFO` 与 `RKNN3_QUERY_ATTENTION_CONFIG` 用于分页 KV Cache 场景，需配合 `RKNN3_QUERY_ALLOCATION_INFO` 枚举各 Core 后使用，详见 3.10 节的 `set_kvcache_pages`。

4.1.4 RKNN3TensorType

张量数据类型枚举：

名称	值	说明
`RKNN3_TENSOR_FLOAT32`	`0`	32位浮点数
`RKNN3_TENSOR_FLOAT16`	`1`	16位浮点数
`RKNN3_TENSOR_INT8`	`2`	8位有符号整数
`RKNN3_TENSOR_UINT8`	`3`	8位无符号整数
`RKNN3_TENSOR_INT16`	`4`	16位有符号整数
`RKNN3_TENSOR_UINT16`	`5`	16位无符号整数
`RKNN3_TENSOR_INT32`	`6`	32位有符号整数
`RKNN3_TENSOR_UINT32`	`7`	32位无符号整数
`RKNN3_TENSOR_INT64`	`8`	64位有符号整数
`RKNN3_TENSOR_UINT64`	`9`	64位无符号整数
`RKNN3_TENSOR_BOOL`	`10`	布尔类型
`RKNN3_TENSOR_INT4`	`11`	4位有符号整数
`RKNN3_TENSOR_FLOAT8E4M3FN`	`12`	8位浮点格式E4M3FN
`RKNN3_TENSOR_BFLOAT16`	`13`	BF16浮点格式
`RKNN3_TENSOR_FLOAT8E8M0`	`14`	8位浮点格式E8M0
`RKNN3_TENSOR_FLOAT4E2M1`	`15`	4位浮点格式E2M1
`RKNN3_TENSOR_TYPE_MAX`	`16`	张量类型最大值

使用说明：应用可根据 `RKNN3TensorAttr.dtype` 选择 NumPy 输入类型。常见映射包括 FP16→ `numpy.float16` 、FP32→ `numpy.float32` 、INT8→ `numpy.int8` 、UINT8→ `numpy.uint8` 、INT32→ `numpy.int32` 。对于 NumPy 不直接支持的运行时格式，通常通过高层接口使用可转换的逻辑数据类型。

4.1.5 RKNN3TensorQntType

张量量化类型枚举：

名称	值	说明
`RKNN3_TENSOR_QNT_NONE`	`0`	无量化
`RKNN3_TENSOR_PER_LAYER_SYMMETRIC`	`1`	每层对称量化
`RKNN3_TENSOR_PER_LAYER_ASYMMETRIC`	`2`	每层非对称量化
`RKNN3_TENSOR_PER_CHANNEL_SYMMETRIC`	`3`	每通道对称量化
`RKNN3_TENSOR_PER_CHANNEL_ASYMMETRIC`	`4`	每通道非对称量化
`RKNN3_TENSOR_PER_GROUP_SYMMETRIC`	`5`	每组对称量化
`RKNN3_TENSOR_PER_GROUP_ASYMMETRIC`	`6`	每组非对称量化

4.1.6 RKNN3TensorLayout

张量布局格式枚举：

名称	值	说明
`RKNN3_TENSOR_UNDEFINED`	`0`	未定义
`RKNN3_TENSOR_NCHW`	`1`	NCHW格式
`RKNN3_TENSOR_NHWC`	`2`	NHWC格式
`RKNN3_TENSOR_NC1HWC2`	`3`	NC1HWC2格式
`RKNN3_TENSOR_CHWN`	`4`	CHWN格式
`RKNN3_TENSOR_HWIO`	`5`	HWIO格式
`RKNN3_TENSOR_OIHW`	`6`	OIHW格式
`RKNN3_TENSOR_O1I1HWI2O2`	`7`	packed Weight 布局

使用说明：

- `NCHW`、`NHWC` 是高层 `inference(data_format=...)` 最常见的逻辑布局。
- `NC1HWC2` 属于运行时内部/原生打包布局。应用通常不直接构造这种打包缓冲区，而是按高层接口要求提供逻辑 NCHW 输入。
- 对 4D 动态 Shape 输入进行匹配时，应同时考虑 NCHW 与 NHWC 两种逻辑表示，避免仅因维度顺序不同误判 Shape 不匹配。

4.1.7 RKNN3MemAllocFlags

内存分配标志枚举：

名称	值	说明
`RKNN3_FLAG_MEMORY_FLAGS_DEFAULT`	`0`	默认内存标志
`RKNN3_FLAG_MEMORY_CACHEABLE`	`1`	可缓存内存
`RKNN3_FLAG_MEMORY_NON_CACHEABLE`	`2`	不可缓存内存

4.1.8 RKNN3MemSyncMode

内存同步模式枚举：

名称	值	说明
`RKNN3_MEMORY_SYNC_TO_DEVICE`	`1`	同步到设备
`RKNN3_MEMORY_SYNC_FROM_DEVICE`	`2`	从设备同步
`RKNN3_MEMORY_SYNC_BIDIRECTIONAL`	`3`	双向同步

4.1.9 RKNN3CoreMask

NPU 核心掩码枚举：

名称	值	说明
`RKNN3_NPU_CORE_AUTO`	`0`	自动选择核心
`RKNN3_NPU_CORE_0`	`1`	核心0
`RKNN3_NPU_CORE_1`	`2`	核心1
`RKNN3_NPU_CORE_2`	`4`	核心2
`RKNN3_NPU_CORE_3`	`8`	核心3
`RKNN3_NPU_CORE_4`	`16`	核心4
`RKNN3_NPU_CORE_5`	`32`	核心5
`RKNN3_NPU_CORE_6`	`64`	核心6
`RKNN3_NPU_CORE_7`	`128`	核心7
`RKNN3_NPU_CORE_ALL`	`0xFFFFFFFF`	自动使用平台支持的 NPU Core

使用说明：Core Mask 为位掩码，每个 bit 表示对应 NPU Core。应用可先获取 `RKNN3_QUERY_CORE_NUMBER`，再根据可用 Core 生成 mask，并检查用户传入 mask 是否与实际 Core 配置一致。

4.1.10 RKNN3KVCachePolicy

KVCache 策略枚举：

名称	值	说明
`RKNN3_KVCACHE_POLICY_DEFAULT`	`0`	默认策略（等同于 RECURRENT）
`RKNN3_KVCACHE_POLICY_RECURRENT`	`1`	循环策略
`RKNN3_KVCACHE_POLICY_NORMAL`	`2`	普通策略，仅使用 max_context_len 长度的 KV Cache
`RKNN3_KVCACHE_POLICY_SAVE_CHECKPOINT`	`0x100`	检查点保存策略，仅对含线性注意力或滑动注意力的模型（如 Qwen3.5 和 Gemma4 系列）生效

使用说明：

- `NORMAL`：常规上下文缓存模式。
- `RECURRENT`：循环复用缓存空间的模式。
- `DEFAULT` 当前等同 recurrent 默认策略。
- `SAVE_CHECKPOINT` 需要结合 `RKNN3KVCachePolicyParam.save_checkpoint` 使用。

4.1.11 RKNN3KVCacheClearPolicy

KVCache 清除策略枚举：

名称	值	说明
`RKNN3_KVCACHE_CLEAR_ALL`	`0`	清除所有
`RKNN3_KVCACHE_KEEP_SYSTEM_PROMPT`	`1`	保留系统提示
`RKNN3_KVCACHE_KEEP_N_TOKENS`	`2`	清除 KVCache，但保留指定数量的 token

使用说明：`CLEAR_ALL` 适合相互独立的请求或在上下文即将耗尽时完全重置；`KEEP_SYSTEM_PROMPT` 适合重复测试/对话中保留固定系统提示缓存，减少重复 Prefill；`KEEP_N_TOKENS` 需要通过 `RKNN3KVCacheClearPolicyParam.keep_n_tokens.n_tokens` 指定保留的 token 数量。

4.1.12 RKNN3LLMInputType

LLM 输入类型枚举：

名称	值	说明
`RKNN3_LLM_INPUT_PROMPT`	`0`	提示输入
`RKNN3_LLM_INPUT_TOKEN`	`1`	标记输入
`RKNN3_LLM_INPUT_EMBED`	`2`	嵌入输入
`RKNN3_LLM_INPUT_MULTIMODAL`	`3`	多模态输入
`RKNN3_LLM_INPUT_AUX`	`4`	辅助输入

使用说明：该枚举对应 `session_run()` 的几种主要输入路径：文本 Prompt、Token ID、预计算 Embed、多模态输入和 AUX 输入。高层接口会根据实际传入参数构造对应底层输入类型。

4.1.13 LLMCallState

LLM 回调状态枚举：

名称	值	说明
`RKLLM_RUN_NORMAL`	`0`	正常运行
`RKLLM_RUN_WAITING`	`1`	等待中
`RKLLM_RUN_FINISH`	`2`	运行完成
`RKLLM_RUN_STOP`	`3`	运行停止
`RKLLM_RUN_MAX_NEW_TOKEN_REACHED`	`4`	达到最大新标记数
`RKLLM_RUN_ERROR`	`5`	运行错误
`RKLLM_RUN_PAUSE`	`6`	运行暂停
`RKLLM_RUN_RESUME`	`7`	运行恢复

使用说明：结果 Callback 应至少处理 NORMAL、FINISH、STOP、MAX_NEW_TOKEN_REACHED 和 ERROR。使用暂停/恢复功能时还应处理 PAUSE、RESUME；流式解码时 WAITING 表示当前 token 片段尚不足以形成完整 UTF-8 字符，可等待后续 token 再输出。

4.1.14 RKNN3GrammarType

Grammar 类型枚举：

名称	值	`grammar_str` 内容	说明
`RKNN3_GRAMMAR_JSON_SCHEMA`	`0`	JSON Schema 文本	约束输出为符合 Schema 的 JSON。适用于信息提取、固定 API 返回和分类结果。
`RKNN3_GRAMMAR_EBNF`	`1`	EBNF 规则文本	使用 EBNF 定义自定义语法，例如固定枚举标签。
`RKNN3_GRAMMAR_BUILTIN_JSON`	`2`	通常为 `None`	使用内置 JSON Grammar，约束输出为任意合法 JSON。
`RKNN3_GRAMMAR_REGEX`	`3`	正则表达式文本	约束输出为匹配正则表达式的字符串，例如日期、订单号或邮箱。
`RKNN3_GRAMMAR_STRUCTURAL_TAG`	`4`	Structural Tag 的 JSON 描述文本	约束由常量文本、JSON Schema 等组合形成的结构，适用于 Tool / Function Calling 等场景。

使用说明：将枚举值传给 `set_grammar()` 的 `grammar_type` 参数。不同类型的 `grammar_str` 格式不能混用；Builtin JSON 不需要传入规则文本。

4.1.15 其他枚举类型

除上述枚举外，当前版本还定义了以下枚举类型：

名称	说明
`RKNN3MemType`	`RKNN3_MEMORY_TYPE_NPU_DRAM = 0 << 0`，`RKNN3_MEMORY_TYPE_EXT_DDR = 1 << 0`
`RKNN3KVCacheDtype`	KV Cache 数据类型，包含 `UNDEFINED`、`INT4_TO_F16`、`INT4_TO_F8`、`INT8_TO_F16`、`FLOAT4_TO_F16`、`FLOAT4_TO_F8`、`FLOAT8_TO_F16`、`FLOAT8_TO_F8`、`FLOAT16`
`RKNN3KVCacheStoreMethod`	KV Cache 存储方式，包含 `UNDEFINED`、`NORMAL`、`GROUP_QUANT`
`RKNN3AttentionType`	注意力类型，包含 `RKNN3_ATTENTION_TYPE_FULL_ATTENTION = 0`、`RKNN3_ATTENTION_TYPE_SLIDING_ATTENTION = 1`、`RKNN3_ATTENTION_TYPE_LINEAR_ATTENTION = 2`
`RKNN3PageCacheType`	分页 KV Cache 的缓存类别，取值与模型 schema 一致，包含 `RKNN3_PAGE_CACHE_GENERAL = 11`、`RKNN3_PAGE_CACHE_FULL_ATTENTION = 12`、`RKNN3_PAGE_CACHE_SLIDING_ATTENTION = 13`、`RKNN3_PAGE_CACHE_LINEAR_ATTENTION = 14`、`RKNN3_PAGE_CACHE_CONV_STREAMING = 15`、`RKNN3_PAGE_CACHE_FULL_RESIDUAL = 17`、`RKNN3_PAGE_CACHE_SLIDING_RESIDUAL = 18`

名称	说明
`LLMOutputCallbackState`	输出回调阶段，包含 `PREFILL_PROCESSING`、`PREFILL_FINISHED`、`DECODE_PROCESSING`、`DECODE_FINISHED`
`RKNN3LLMTaskType`	LLM 任务类型，包含 `RKNN3_LLM_TASK_GENERATE = 0`、`RKNN3_LLM_TASK_EMBEDDING = 1`、`RKNN3_LLM_TASK_RERANKER = 2`
`RKNN3ImFmt`	图像格式，包含 `RGB888`、`BGR888`、`GRAY8`、`YCbCr/YCrCb 420/422 SP`、`UNKNOWN`
`RKNN3ImProcFlag`	图像处理标志，包含 `CROP`、`RESIZE`、`FILL`、`COLOR_SPACE_CONVERT`、`DECODE`、`ENCODE`
`RKNN3OpTargetType`	自定义算子目标类型，目前包含 `RKNN3_OP_TARGET_TYPE_CPU = 1`
`RKNN3OpPluginType`	自定义算子插件类型，包含 `POSTPROCESS` 和 `CUSTOM_OP`

`LLMOutputCallbackState` 使用说明：

- `PREFILL_PROCESSING`：Prefill 阶段正在执行；
- `PREFILL_FINISHED`：Prefill 阶段已完成，适合读取仅在 Prefill 结束后才有效的输出；
- `DECODE_PROCESSING`：Decode 阶段正在执行；
- `DECODE_FINISHED`：Decode 阶段已完成。

应用应根据需要的阶段判断 `state` 后再读取 `LLMOutputCallback` 提供的输出张量，而不是假设每次 Callback 中的数据含义都相同。

4.2 通用结构

这些结构大多由 `rknn3_query()` 返回，用于描述模型 I/O 数量、动态 Shape、SDK/设备和运行时配置。应用通常只读取其字段，不直接修改运行时内部数据。

4.2.1 RKNN3InputOutputNum

输入输出数量信息：

字段	类型	说明
`n_input`	`uint32`	输入数量
`n_output`	`uint32`	输出数量

使用说明：由 `RKNN3_QUERY_IN_OUT_NUM` 返回。应用通常先读取 `n_input` / `n_output`，再按索引循环查询每个 `RKNN3TensorAttr`。

4.2.2 RKNN3PerfDetail

性能详情：

字段	类型	说明
`perf_data`	`char*`	性能数据
`data_len`	`uint64`	数据长度

4.2.3 RKNN3PerfRun

运行性能：

字段	类型	说明
`run_duration`	`int64`	运行时长

4.2.4 RKNN3SDKVersion

SDK版本：

字段	类型	说明
`api_version`	`char[256]`	API 版本
`drv_version`	`char[256]`	驱动版本

4.2.5 RKNN3CustomString

自定义字符串：

字段	类型	说明
`string`	`char[1024]`	字符串

4.2.6 RKNN3Config

RKNN3配置：

字段	类型	说明
`priority`	`int32`	优先级
`run_timeout`	`uint32`	运行超时
`run_core_mask`	`uint32`	运行核心掩码
`user_mem_weight`	`uint8`	用户内存权重
`user_mem_internal`	`uint8`	用户内部内存
`user_sram`	`uint8`	用户SRAM

4.2.7 RKNN3ShapeInfo

形状信息：

字段	类型	说明
`shape_id`	`int32`	形状 ID
`n_inputs`	`uint32`	输入数量
`input_attrs`	`RKNN3TensorAttr*`	输入属性
`n_outputs`	`uint32`	输出数量
`output_attrs`	`RKNN3TensorAttr*`	输出属性
`is_default`	`uint8`	是否默认

使用说明：描述一个动态 Shape 组合及其输入/输出属性。应用可读取其中的 `shape_id` 和输入 Shape，并与实际输入 Shape 做匹配。

4.2.8 RKNN3ShapeConfig

形状配置：

字段	类型	说明
`n_shapes`	`uint32`	形状数量
`current_shape_id`	`int32`	当前形状 ID

使用说明：用于描述模型拥有多少组动态 Shape 以及当前激活的 Shape。高层 `get_dynamic_shape_infos()` 会把底层动态 Shape 查询结果整理成更方便 Python 使用的结构。

4.2.9 RKNN3InitExtend

扩展初始化参数：

字段	类型	说明
`device_id`	`char*`	设备 ID
`reserved`	`uint8[128]`	保留

4.2.10 RKNN3NodeMemInfo

节点内存信息：

字段	类型	说明
`total`	`uint64`	总内存
`free`	`uint64`	可用内存

4.2.11 RKNN3DevMemInfo

设备内存信息：

字段	类型	说明
`node_num`	`uint32`	节点数量
`sys_total`	`uint64`	系统总内存
`sys_free`	`uint64`	系统可用内存
`node_mem_info`	`RKNN3NodeMemInfo[128]`	节点内存信息

使用说明：可通过 `RKNN3_QUERY_DEVICE_MEM_INFO` 读取 `sys_total/sys_free` 和每个 node 的内存信息，用于运行前检查设备可用内存或诊断 OOM。

4.2.12 RKNN3Device

设备信息：

字段	类型	说明
`id`	`char[64]`	设备 ID
`type`	`char[64]`	设备类型
`mem_info`	`RKNN3DevMemInfo`	内存信息

4.2.13 RKNN3Devices

设备列表：

字段	类型	说明
`n_devices`	`uint32`	设备数量
`devices`	`RKNN3Device[64]`	设备信息

4.2.14 自定义算子结构

名称	说明
`RKNN3CustomOpContext`	自定义算子上下文，字段包括 `rknn_ctx`、`priv_data`、`user_data`
`RKNN3CustomOp`	自定义算子注册信息，字段包括 `op_type`、`plugin_type`、`target`、`version`、`author`、`description` 以及 `init/prepare/compute/deinit/get_output_num/get_attrs` 等回调

4.3 张量与内存结构

张量结构用于连接“张量属性”和“实际内存”。普通高层推理只需 `numpy.ndarray`；显式内存和 Callback 场景则需要结合 `RKNN3TensorAttr`、`RKNN3TensorMemory` 与 `RKNN3Tensor` 使用。

4.3.1 RKNN3QuantInfo

量化信息：

字段	类型	说明
`scale`	`float`	缩放因子
`zero_point`	`int32`	零点

4.3.2 RKNN3TensorAttr

张量属性信息：

字段	类型	说明
`index`	`uint32`	索引
`name`	`char[256]`	名称
`n_dims`	`uint32`	维度数
`shape`	`uint32[16]`	形状
`aligned_size`	`uint64`	对齐大小
`n_stride`	`uint32`	步长维度数
`stride`	`uint64[17]`	步长
`n_elems`	`uint32`	元素数量
`dtype`	`int`	数据类型
`layout`	`int`	布局
`qnt_type`	`int`	量化类型
`qnt_info`	`RKNN3QuantInfo`	量化信息
`n_orig_dims`	`uint32`	原始张量维度数
`orig_shape`	`uint32[16]`	原始张量形状
`orig_layout`	`int`	原始张量布局
`core_id`	`int32`	核心 ID

使用说明：

- `shape` 只有前 `n_dims` 项有效；`stride` 同理应结合 `n_stride` /实际维度读取；
- `aligned_size` 是运行时按对齐要求计算的缓冲区大小，显式创建内存时应优先使用该值，而不是简单按元素数相乘；

- `dtype`、`layout`、`qnt_type/qnt_info` 决定数据如何解释；准备输入数据、动态 Shape 匹配和性能分析时都应以这些字段为依据；
- `core_id` 表示张量对应的 NPU Core，在多 Core 用户内存管理中用于选择目标 Core；
- 对于 Output Callback，通常先通过 `RKNN3_QUERY_OUTPUT_ATTR` 获取该结构，再据此调用 `create_mem()` 为对应输出张量准备缓冲区。

4.3.3 RKNN3MemSize

内存大小：

字段	类型	说明
`total_const_size`	`uint64`	常量总大小
`total_internal_size`	`uint64`	内部总大小
`total_dma_allocated_size`	`uint64`	DMA分配总大小
`total_sram_size`	`uint64`	SRAM总大小
`free_sram_size`	`uint64`	可用SRAM大小

4.3.4 RKNN3TensorMemory

张量内存：

字段	类型	说明
`virt_addr`	`void*`	虚拟地址
`phys_addr`	`uint64`	物理地址
`fd`	`int32`	文件描述符
`buffer_id`	`uint64`	缓冲区 ID
`offset`	`uint64`	偏移
`size`	`uint64`	大小
`flags`	`uint64`	标志
`core_id`	`int32`	核心 ID
`priv_data`	`void*`	私有数据
`mem_type`	`int`	内存类型

使用说明：

- `virt_addr` 用于 CPU 侧直接访问张量缓冲区，`size` 表示实际分配的字节数；

- `phys_addr`、`fd`、`offset` 用于物理内存、DMA-BUF/共享内存等外部内存场景；
- Output Callback 中通常通过 `RKNN3Tensor.mem` 取得该结构，再根据 `attr.dtype` 将 `virt_addr` 转换为对应的数据类型进行读取；
- 由 `create_mem()` 创建的内存使用完成后应调用 `destroy_mem()`，不要直接释放 `virt_addr`。

4.3.5 RKNN3Tensor

RKNN3张量：

字段	类型	说明
`mem`	`RKNN3TensorMemory*`	内存
`attr`	`RKNN3TensorAttr*`	属性

使用说明：`RKNN3Tensor` 本身不直接保存张量数据，而是通过两个指针把元信息与缓冲区关联起来：

- `attr` 指向 `RKNN3TensorAttr`，用于解释输出名称、Shape、dtype、Layout 等；
- `mem` 指向 `RKNN3TensorMemory`，用于访问实际数据；
- `LLMOutputCallback` 收到的 `output_tensors` 即为该结构数组，因此回调中通常同时读取 `tensor.attr.contents` 和 `tensor.mem.contents`。

4.3.6 RKNN3AllocationInfo

内存分配信息：

字段	类型	说明
`core_id`	`int32`	核心 ID
`command_mem`	`RKNN3TensorMemory`	命令内存
`weight_mem`	`RKNN3TensorMemory`	权重内存
`internal_mem`	`RKNN3TensorMemory`	内部内存
`kvcache_mem`	`RKNN3TensorMemory`	KVCache 内存
`reserved`	`uint8[128]`	保留

4.3.7 RKNN3CoreMemSize

名称	说明
`RKNN3CoreMemSize`	核心内存大小信息，字段包括 `core_id`、`weight_size`、`internal_size`、`reserved`

4.4 LLM 和 VLM 结构

这些结构覆盖 LLM 参数、生成结果、运行状态以及多模态输入。LLM 场景通常会直接使用 `RKNN3LLMConfig`、`RKLLMResult`、`RKLLMRunState`；VLM 场景则可通过 `Float16` 指针把视觉模型输出 embedding 传给多模态输入结构。

4.4.1 Float16

16位浮点数表示：

字段	类型	说明
`frac`	`uint16, 10位`	尾数
`exp`	`uint16, 5位`	指数
`sign`	`uint16, 1位`	符号

使用说明：VLM 场景可将 NumPy `float16` 输出通过 `outputs.ctypes.data_as(ctypes.POINTER(Float16))` 转成 `Float16*`，再赋给 `RKNN3Image.image_embed`。因此底层指针有效期间，对应 NumPy 数组必须保持存活。

4.4.2 RKNN3VocabInfo

词汇表信息：

字段	类型	说明
`vocab_size`	`int`	词汇表大小
`special_bos_id`	`int[64]`	特殊开始标记 ID
`special_eos_id`	`int[64]`	特殊结束标记 ID
`n_special_bos_id`	`int`	特殊开始标记数量
`n_special_eos_id`	`int`	特殊结束标记数量
`linefeed_id`	`int`	换行符 ID
`skip_special_token`	`bool`	是否跳过特殊标记
`ignore_eos_token`	`bool`	是否忽略结束标记
`reserved`	`uint8[64]`	保留

使用说明：`special_eos_id`、`skip_special_token`、`ignore_eos_token` 等字段直接影响生成终止和输出处理。性能测试场景可启用 `ignore_eos_token` 强制生成到 `max_new_tokens`，以便稳定测量 Decode 性能。

4.4.3 RKNN3SamplingParams

采样参数：

字段	类型	说明
`top_k`	`int32`	top-k
`top_p`	`float`	top-p
`temperature`	`float`	温度
`repeat_penalty`	`float`	重复惩罚
`frequency_penalty`	`float`	频率惩罚
`presence_penalty`	`float`	存在惩罚

使用说明：`top_k/top_p/temperature` 控制采样范围与随机性，`repeat_penalty/frequency_penalty/presence_penalty` 控制重复倾向。需要获得更确定的输出时，可减小采样随机性，例如设置较小的 `top_k`；性能测试和评测场景通常也会采用更确定的采样配置。

4.4.4 RKNN3LLMTensor

LLM 张量输入：

字段	类型	说明
`name`	`char*`	名称
`prompt`	`char*`	提示
`embed`	`Float16*`	嵌入
`tokens`	`int32*`	标记
`n_tokens`	`uint64`	标记数量
`enable_thinking`	`bool`	启用思考

使用说明：该结构承载纯文本/Token/Embedding 路径的底层数据。高层 `session_run(prompt=...)`、`session_run(tokens=...)` 或 `session_run(embeds=...)` 最终会构造与此对应的输入；三种主输入不应同时设置。

4.4.5 RKNN3AuxTensor

辅助张量（与RKNN3Tensor 相同）

使用说明：`RKNN3AuxTensor` 与普通 `RKNN3Tensor` 等价，表示额外模型输入张量。Python 高层更常通过 `RKNN3AuxTensorWrapper` 传入 NumPy 数据，由封装层构造底层 AUX tensor。

4.4.6 RKNN3LLMMultiModelTensor

多模态张量输入：

字段	类型	说明
`name`	`char*`	名称
`prompt`	`char*`	提示
`tokens`	`int32*`	标记
`n_tokens`	`uint64`	标记数量
`enable_thinking`	`bool`	启用思考
`image`	`RKNN3Image`	图像数据
`audio`	`RKNN3Audio`	音频数据
`video`	`RKNN3Video`	视频数据

4.4.7 RKNN3LLMInputUnion

LLM 输入联合：

字段	类型	说明
`llm_input`	`RKNN3LLMTensor`	LLM 输入
`multimodal_input`	`RKNN3LLMMultiModelTensor`	多模态输入
`aux_input`	`RKNN3AuxTensor`	辅助输入

4.4.8 RKNN3LLMInput

LLM 输入：

字段	类型	说明
`role`	`char*`	角色
`input_type`	`int`	输入类型
`input_union`	`RKNN3LLMInputUnion`	输入联合

4.4.9 RKNN3LLMExtendParam

扩展 LLM 参数：

字段	类型	说明
`reserved`	`uint8[128]`	保留

4.4.10 RKNN3LLMParam

LLM 参数：

字段	类型	说明
`logits_name`	`char*`	logits名称
`max_context_len`	`int32`	最大上下文长度
`sampling_param`	`RKNN3SamplingParams`	采样参数
`vocab_info`	`RKNN3VocabInfo`	词汇表信息
`extend_param`	`RKNN3LLMExtendParam`	扩展参数

使用说明：初始化 Session 时，Python 字典中的采样参数、词表信息和上下文长度会被转换到该结构。`max_context_len` 建议以 `RKNN3LLMConfig.max_ctx_len` 为基准；当设置值超出模型支持范围时，应在应用侧进行校验并停止初始化。

4.4.11 RKNN3LLMInferParam

LLM 推理参数：

字段	类型	说明
`keep_history`	`int`，1 表示保留，0 表示不保留	是否保留历史上下文
`max_new_tokens`	`int32`	最大新生成 token 数
`prefill_only`	`bool`	是否仅执行一次 Prefill
`disable_sampling`	`bool`	是否在单次 Prefill 后禁用采样
`keep_grammar`	`bool`	是否在多次推理之间保留 grammar matcher 状态。默认值为 false。启用后，每次 `rknn3_session_run()` 前后不再重新初始化 grammar matcher，grammar 状态可跨多次 run 保留，用于连续受约束生成
`reserved`	`uint8[127]`	保留

使用说明：`session_run()` 的 `keep_history`、`max_new_tokens`、`prefill_only`、`disable_sampling`、`keep_grammar` 参数可覆盖会话默认推理参数。评测任务通常 `keep_history=False`；连续对话通常为 True。Python 高层接口中这些布尔参数支持传入 `0` / `1` 等整型值，传入后会自动转换为 `bool` 再设置到该结构。

4.4.12 RKLLMResult

LLM 生成结果：

字段	类型	说明
`token_ids`	`int*`	标记 ID
`num_tokens`	`int`	标记数量

使用说明：在 `LLMResultCallback` 中通过 `result_ptr.contents.num_tokens` 获取本次回调 token 数量，再从 `token_ids` 读取 token。流式输出时可累计 token 后统一调用 Tokenizer decode，以避免单个 token 造成不完整 UTF-8 字符。

4.4.13 RKLLMRunState

LLM 运行状态：

名称	说明
`n_total_tokens`	当前已处理的 token 总数（`uint64`）
`n_reuse_tokens`	当前输入从 KV Cache 复用的 token 数量（`uint64`）
`n_max_tokens`	可处理的最大 token 数量（`uint64`）
`n_prefill_tokens`	Prefill 阶段处理的 token 数量（`uint64`）
`n_decode_tokens`	Decode 阶段生成的 token 数量（`uint64`）
`n_input_tokens`	输入 token 数量（`uint64`，与 `n_prefill_tokens` 一致）
`n_output_tokens`	输出 token 数量（`uint64`，通常为 `n_decode_tokens + 1`）
`input_tokens`	输入 token 数组指针（`int32*`），由运行时管理，用户不应释放
`output_tokens`	输出 token 数组指针（`int32*`），由运行时管理，用户不应释放
`kvcache_policy`	KVCache 策略（`int`）
`n_loras_enabled`	启用 LoRA 数量（`int32`）
`loras_enabled`	启用的 LoRA 数组（`RKNN3Lora*`）

使用说明：

- `n_total_tokens/n_max_tokens` 可用于判断剩余上下文空间。
- `n_prefill_tokens/n_decode_tokens` 可用于性能统计或校验本次运行阶段。
- `n_reuse_tokens` 反映当前输入从 KVCache 复用的 token 数量。
- `kvcache_policy`、`n_loras_enabled/loras_enabled` 可用于确认当前 Session 的缓存和 LoRA 配置。
- `input_tokens/output_tokens` 由 Runtime 管理，用户只读，不应自行释放。

4.4.14 RKLLMResultLastHiddenLayer

最后隐藏层结果：

字段	类型	说明
`hidden_states`	`float*`	隐藏状态
`embd_size`	`int`	嵌入大小
`num_tokens`	`int`	标记数量

4.4.15 RKNN3LLMConfig

名称	说明
`RKNN3LLMConfig`	LLM 模型配置，字段包括 `chat_template`、`vocab_size`、`embedding_dim`、`max_ctx_len`、`max_position_embeddings`、`kvcache_store_method`、`kvcache_dtype`、`kvcache_group_size`、`kvcache_residual_depth`、`model_type`、`task_type`、`rope_cache_host_storage`、`n_attention_kvcache_lens`、`attention_kvcache_lens`、`n_attention_configs`、`reserved`

使用说明：这是 LLM 应用最重要的模型只读配置之一。建议在 Session 初始化前查询该结构，并根据其中字段构造运行参数：

- `vocab_size`：校验 tokenizer 词表并重塑 embedding table；
- `embedding_dim`：校验或推导 token embedding 维度；
- `max_ctx_len`：作为 Session 的 `max_context_len` 基准，应用设置值不应超过模型支持范围；
- `max_position_embeddings`：表示模型位置编码支持的最大位置范围，可用于检查长上下文配置；
- `model_type`、`task_type`：识别模型类型以及 Generate/Embedding/Reranker 等任务；
- `kvcache_store_method`、`kvcache_dtype`、`kvcache_group_size`、`kvcache_residual_depth`：描述模型 KVCache 的存储与量化配置；
- `rope_cache_host_storage`：表示模型是否需要应用侧参与 Host RoPE Cache 管理。该值非 0 时，部分模型可能存在需要通过 `LLMInputCallback` 按当前位置填充的 RoPE 扩展输入；
- `n_attention_kvcache_lens`、`attention_kvcache_lens`：描述不同注意力类型对应的 KVCache 长度配置；
- `n_attention_configs`：模型包含的注意力配置条目数量。分页 KV Cache 模型可通过该值确定 `RKNN3_QUERY_ATTENTION_CONFIG` 需要查询的数组长度。

注意：扩展输入的具体名称、数据格式和 Host Cache 文件组织方式由模型决定，不属于 RKNN3Lite 通用 API 的固定约定。

4.4.16 RKNN3GrammarConfig

Grammar 引擎词表配置：

字段	类型	说明
`vocab_size`	`int32`	词表大小，即 token piece 数量。
`encoded_vocab`	`uint8*`	按 token ID 顺序编码的词表数据。每个 token 由一个 `int32` 长度和对应 token bytes 顺序组成。
`encoded_vocab_len`	`int32`	`encoded_vocab` 的总字节数。
`n_stop_tokens`	`int32`	停止 token 数量。
`stop_tokens`	`int32*`	停止 token ID 数组。

通常无需手动填充此结构。推荐把按 token ID 排列的 `token_pieces` 传给 `init_grammar()`，由 Python API 自动构造：

```
from rknn3lite.api.rknn3_types import RKNN3GrammarConfig

config = RKNN3GrammarConfig.from_token_pieces(
    token_pieces,
    stop_tokens=[tokenizer.eos_token_id],
)
ret = rknn_lite.init_grammar(config=config)
```

`from_token_pieces()` 支持 `str` 和 bytes-like token piece；`str` 按 UTF-8 编码。手动构造时必须确保 `encoded_vocab` 和 `stop_tokens` 指向的缓冲区在当前 Context 使用 Grammar 的整个期间保持有效。使用 `from_token_pieces()` 构造的结构会自动保持其内部缓冲区引用。

4.4.17 RKNN3GrammarSetOptions

Grammar 设置选项：

字段	类型	说明
`any_whitespace`	`int`	空白字符处理选项。
`indent`	`int`	格式化缩进选项。
`strict_mode`	`int`	严格模式选项。
`max_whitespace_cnt`	`int`	最大连续空白字符数量选项。
`any_order`	`int`	JSON 对象键顺序处理选项。
`item_separator`	`char*`	项分隔符。
`key_value_separator`	`char*`	键值分隔符。
`root_rule_name`	`char*`	EBNF 根规则名称。

通过 `RKNN3GrammarSetOptions.default()` 可创建带有 Runtime 默认初始化值的选项对象，并按需覆盖字段：

```
from rknn3lite.api.rknn3_types import (
    RKNN3GrammarSetOptions,
    RKNN3GrammarType,
)

options = RKNN3GrammarSetOptions.default(
    root_rule_name='root',
)
ret = rknn_lite.set_grammar(
    'root ::= "yes" | "no"\\n',
    RKNN3GrammarType.RKNN3_GRAMMAR_EBNF,
    options=options,
)
```

字符串字段支持 `str`、`bytes` 或 `None`，其中 `str` 会自动使用 UTF-8 编码。若不需要调整 Grammar 引擎的格式选项，建议在 `set_grammar()` 中保留 `options=None`。

4.5 KVCache 结构

KVCache 结构用于描述缓存策略参数及不同注意力类型/长度分组的缓存配置。普通应用通常只选择策略；需要 recurrent、checkpoint 或多长度分组时才需要进一步填写或查询这些结构。

4.5.1 RECURRENT

循环 KVCache 参数：

字段	类型	说明
`n_keep`	`int64`	循环策略下需要保留的 KV Cache 数量
`n_keep_aligned`	`int64`	按 `kvcache_group_size` 对齐后的保留数量

注：模型包含系统提示时，运行时会自动使用系统提示长度，此时 `n_keep` 和 `n_keep_aligned` 不生效。

使用说明：仅在选择 recurrent 类 KVCache 策略并需要自定义保留长度时使用。`n_keep_aligned` 应按模型 `kvcache_group_size` 对齐；含系统提示的模型通常由 Runtime 自动保留系统提示部分。

4.5.2 RKNN3KVCachePolicyParam

KVCache 策略参数：

名称	说明
`recurrent`	循环参数（`RECURRENT`）
`save_checkpoint`	检查点保存参数（SAVE_CHECKPOINT），仅对含线性注意力或滑动注意力的模型生效
`reserved`	保留（`uint8[64]`）

SAVE_CHECKPOINT 子结构字段：

名称	说明
`checkpoint_start_pos`	开始保存检查点的位置（int64），需对齐到 128；未对齐时运行时会强制调整
`checkpoint_interval`	保存检查点的 token 间隔（int64），需对齐到 128；未对齐时运行时会强制调整
`max_checkpoint_count`	最大检查点保存数量（int64），应小于 `(max_context_len - checkpoint_start_pos) / checkpoint_interval`；超出时运行时会调整为允许的最大值
`checkpoint_tail_overwrite`	是否将最后一个检查点槽位作为滚动覆盖槽位（bool）。设为 True 时，前 `max_checkpoint_count - 1` 个槽位固定保存，后续超出范围的检查点持续覆盖最后一个槽位

4.5.3 RKNN3KVCacheClearPolicyParam

KVCache 清理策略参数：

名称	类型	说明
`keep_n_tokens`	`RKNN3KVCacheClearPolicyParamKeepNTokens`	`RKNN3_KVCACHE_KEEP_N_TOKENS` 的参数
`reserved`	`uint8[64]`	保留字段

`keep_n_tokens` 子结构字段：

名称	类型	说明
`n_tokens`	`uint32`	清理 KVCache 时保留的 token 数量

示例：

```
clear_param = RKNN3KVCacheClearPolicyParam()
clear_param.keep_n_tokens.n_tokens = 128
rknn_lite.clear_kvcache(
    RKNN3KVCacheClearPolicy.RKNN3_KVCACHE_KEEP_N_TOKENS,
    param=clear_param,
)
```

4.5.4 RKNN3AttentionKvCacheLens

名称	说明
`RKNN3AttentionKvCacheLens`	注意力类型对应的 KV Cache 缓冲区长度配置，字段包括 `attention_type`、`n_kvcache_buffer_lens`、`kvcache_buffer_lens`；`kvcache_buffer_lens` 最多包含 `RKNN3_MAX_KVCACHE_LEN_GROUPS`（16）项

4.5.5 RKNN3KvCacheLenGroupInfo

名称	说明
`RKNN3KvCacheLenGroupInfo`	每核 KV Cache 长度分组信息（用于 `RKNN3_QUERY_KVCACHE_LEN_GROUP_INFO`），字段包括 `core_id`、`n_groups`、`active_group_id`、`kvcache_sizes`；`n_groups = 0` 表示兼容旧版单分组模式，`kvcache_sizes[group_id]` 表示该核对应分组的 KV Cache 大小

使用说明：由 `RKNN3_QUERY_KVCACHE_LEN_GROUP_INFO` 返回。`n_groups` 表示可选长度分组数，`active_group_id` 表示当前分组，`kvcache_sizes[group_id]` 给出对应 Core 在该组下的缓存大小，可在调用 `set_kvcache_group()` 前用于合法性检查。

4.5.6 RKNN3AttentionConfig

注意力配置，用于描述模型单个注意力分支的 KVCache、位置编码和多模态位置编码（mRoPE）参数：

字段	类型	说明
`mask_name`	`char[256]`	mask 名称
`kvcache_buffer_len`	`int32`	KVCache 缓冲区长度
`kvcache_realtime`	`bool`	KVCache 是否实时
`kvcache_dtype`	`int`	KVCache 数据类型（`RKNN3KVCacheDtype`）
`kvcache_store_method`	`int`	KVCache 存储方式（`RKNN3KVCacheStoreMethod`）
`kvcache_group_size`	`int32`	KVCache 分组大小
`kvcache_residual_depth`	`int32`	KVCache 残差深度
`sliding_window_size`	`int32`	滑动窗口大小
`attention_type`	`int`	注意力类型（`RKNN3AttentionType`）
`position_name`	`char[256]`	位置编码名称
`max_position_embeddings`	`int32`	最大位置编码范围
`position_embeddings_host_storage`	`bool`	位置编码是否存放于 Host
`mrope_type`	`char[256]`	mRoPE 类型
`mrope_section`	`int32[8]`	mRoPE section
`n_mrope_section`	`int32`	mRoPE section 有效元素数量
`mrope_new_id_name`	`char[256]`	mRoPE 新 ID 名称
`kvcache_buffer_lens`	`int32[16]`	KVCache 缓冲区长度列表
`n_kvcache_buffer_lens`	`int32`	`kvcache_buffer_lens` 有效元素数量
`tokens_per_page`	`uint32`	每个 page 承载的 token 数量

字段	类型	说明
`rollback_tokens_align`	`uint32`	回滚 token 对齐粒度

使用说明：由 `RKNN3_QUERY_ATTENTION_CONFIG` 返回，数组长度取自 `RKNN3_QUERY_LLM_CONFIG` 的 `n_attention_configs`。分页 KV Cache 场景下，`tokens_per_page` 用于把上下文长度换算成所需 page 数量；`attention_type` 为 `RKNN3_ATTENTION_TYPE_FULL_ATTENTION` 的条目是分页缓存的主要配置来源。

4.5.7 RKNN3CachePageAttr

单个缓存类别的 page 属性：

字段	类型	说明
`cache_type`	`int32`	缓存类别（`RKNN3PageCacheType`）
`page_size`	`uint64`	单个 page 的字节数
`max_page_num`	`uint32`	编译期确定的 page 容量上限，不随绑定或增长变化
`reserved`	`uint8[32]`	保留字段

使用说明：由 `RKNN3KvCachePageInfo.caches` 提供。分配 page 时，`page_size` 作为 `create_mem()` 的字节大小，`max_page_num` 作为该类别可绑定的最大 page 数量。

4.5.8 RKNN3KvCachePageInfo

分页 KV Cache 的模型侧几何信息，用于 `RKNN3_QUERY_KVCACHE_PAGE_INFO`：

字段	类型	说明
`core_id`	`int32`	输入参数，指定物理 Core ID
`max_request_num`	`uint32`	连续批处理支持的最大并发请求数
`n_cache_types`	`uint32`	有效缓存类别数量。非分页模型返回 0
`caches`	`RKNN3CachePageAttr[16]`	各缓存类别的 page 属性
`reserved`	`uint8[128]`	保留字段，供后续扩展

使用说明：调用前需设置 `core_id`，该值可由 `RKNN3_QUERY_ALLOCATION_INFO` 返回的 `RKNN3AllocationInfo.core_id` 获得。`n_cache_types` 为 0 表示当前模型不使用分页 KV Cache，应改用 `set_kvcache_mem()`。

4.5.9 RKNN3KvCachePageBinding

分页 KV Cache 的绑定条目，作为 `set_kvcache_pages()` 的输入：

字段	类型	说明
`core_id`	`int32`	物理 Core ID
`cache_type`	`int32`	缓存类别（`RKNN3PageCacheType`）
`request_index`	`uint32`	请求索引
`n_pages`	`uint32`	page 数量（前缀长度）；为 0 表示解绑该条目
`pages`	`RKNN3TensorMemory**`	page 指针数组，每个元素为 `core_id` 上一个 `page_size` 字节的分配
`reserved`	`uint8[64]`	保留字段

使用说明：`pages` 指向的指针数组由调用方持有，必须保证在 `set_kvcache_pages()` 调用期间及绑定有效期内不被回收。

4.6 LoRA 结构

LoRA 相关 Python API 以 `RKNN3Lora` 作为适配器描述对象。该对象既用于 `query_lora()` 返回信息，也作为 load/enable/disable/unload 等接口的参数。

4.6.1 RKNN3Lora

LoRA 配置：

字段	类型	说明
`lora_name`	`char[256]`	LoRA 名称
`scale`	`float`	缩放因子

使用说明：

- `lora_name` 标识适配器，通常直接使用 `query_lora()` 返回的名称。
- `scale` 控制 LoRA 对当前推理的作用强度。可在启用前修改该字段，再通过 `enable_lora()` 应用到指定 Session。
- 同一个 `RKNN3Lora` 可在不同 Session 上分别启用/禁用。

4.7 图像、音频和视频结构

`RKNN3Image`、`RKNN3Audio` 和 `RKNN3Video` 描述的是已经提取完成的多模态 embedding 及对应特殊标记，不是原始媒体文件本身。视觉模型与 LLM 解耦时，应用先运行视觉模型，再将输出 embedding 填入这些结构。

4.7.1 RKNN3Image

图像输入：

字段	类型	说明
`image_embed`	`Float16*`	图像嵌入
`n_image_tokens`	`uint64`	图像标记数量
`n_image`	`uint64`	图像数量
`image_start`	`char*`	图像开始标记
`image_end`	`char*`	图像结束标记
`image_content`	`char*`	图像内容标记
`image_width`	`uint64`	图像宽度
`image_height`	`uint64`	图像高度

使用说明：VLM 场景的典型构造方式是：

```
image = RKNN3Image()
image.image_embed = vision_output.ctypes.data_as(ctypes.POINTER(Float16))
image.n_image_tokens = vision_output.shape[1]
image.n_image = vision_output.shape[0]
image.image_width = image_size
image.image_height = image_size
image.image_start = b"<|vision_start|>"
image.image_end = b"<|vision_end|>"
image.image_content = b"<|image_pad|>"
```

`image_start/image_end/image_content` 必须与具体模型 Tokenizer/Chat Template 的多模态特殊标记一致；`image_embed` 指向的 NumPy 缓冲区在 `session_run()` 完成前必须保持有效。

4.7.2 RKNN3Audio

音频输入：

字段	类型	说明
`audio_embed`	`Float16*`	音频嵌入
`n_audio_tokens`	`uint64`	音频标记数量
`n_audio`	`uint64`	音频数量
`audio_start`	`char*`	音频开始标记
`audio_end`	`char*`	音频结束标记
`audio_content`	`char*`	音频内容标记

使用说明：字段组织方式与 `RKNN3Image` 类似，核心是提供音频 embedding、每段音频 token 数量、音频数量以及模型约定的开始/结束/内容特殊标记。传入的是特征 embedding，不是 PCM/WAV 文件路径。

4.7.3 RKNN3Video

视频输入：

字段	类型	说明
`video_embed`	`Float16*`	视频嵌入
`n_frame_tokens`	`uint64`	帧标记数量
`n_frame_per_video`	`uint64`	每视频帧数量
`n_video`	`uint64`	视频数量
`video_start`	`char*`	视频开始标记
`video_end`	`char*`	视频结束标记
`video_content`	`char*`	视频内容标记
`frame_width`	`uint64`	帧宽度
`frame_height`	`uint64`	帧高度

使用说明：用于一次传递一个或多个视频的帧级 embedding。`n_frame_tokens`、`n_frame_per_video` 和 `n_video` 共同描述 embedding 的逻辑组织，特殊标记同样必须与模型约定一致。

4.7.4 RKNN3ImRect

名称	说明
`RKNN3ImRect`	图像矩形区域，字段包括 `x`、`y`、`width`、`height`

4.7.5 RKNN3ImMetadata

名称	说明
`RKNN3ImMetadata`	图像元数据，字段包括 `peer_im_mem_addr`

4.7.6 RKNN3ImMem

名称	说明
`RKNN3ImMem`	图像内存，字段包括 `width`、`height`、`stride`、`format`、`sync_to_host`、`data_mem`、`metadata`

4.8 Callback 类型

Python 侧可通过 `ctypes.CFUNCTYPE` 构造回调，并将其写入 `RKLLMCallback`。由于底层 Runtime 会在后续推理过程中调用这些 Python 对象，回调函数对象、userdata 以及被引用的 Tokenizer/Embedding 数据都必须在 Session 生命周期内保持有效，不能被 Python 垃圾回收。

4.8.1 LLMResultCallback

LLM 结果回调函数类型，用于处理 LLM 推理生成的结果。

```
typedef int (*LLMResultCallback)(void* userdata, RKLLMResult* result, LLMCallState
state)
```

参数：

名称	说明
`userdata`	用户自定义数据指针，在注册回调时传入
`result`	指向 LLM 推理结果的指针（RKLLMResult），包含生成的 token ID 及数量
`state`	LLM 调用状态（LLMCallState），如运行中、完成、错误等

返回值：成功返回 0，失败返回非 0

使用说明：流式输出时不建议直接对每次回调的单个 token 做最终文本输出，而应累计 token 后解码，并在 FINISH/STOP/MAX_NEW_TOKEN_REACHED 时补齐剩余文本。这样可避免 UTF-8 多字节字符被拆分时出现 `` 。Callback 内还可根据业务条件调用 `session_stop()` 。

4.8.2 LLMSamplingCallback

采样回调函数类型，用于在 LLM 推理过程中自定义采样策略，处理 logits 并返回选中的 token ID。

```
typedef int (*LLMSamplingCallback)(void* userdata, Float16* logits, char*
logits_name)
```

参数：

名称	说明
`userdata`	用户自定义数据指针，在注册回调时传入
`logits`	指向当前输出层 logits 数组的指针（`Float16` 类型），数组长度为模型词表大小
`logits_name`	当前 logits 对应的输出层名称；模型只有单个输出时可为空

返回值：成功时返回选中的 token ID（0 为合法值，如词表中第一个 token），失败时返回负数错误码

使用说明：该回调为可选实现，仅在需要自定义采样策略时注册，注册后采样过程由该回调负责完成。回调内应根据 `logits` 自行完成采样并返回选中的 token ID，返回负值表示采样失败。多输出模型可通过 `logits_name` 区分不同输出层的 logits。

4.8.3 LLMGetEmbedCallback

获取嵌入回调函数类型，用于获取指定 token 对应的嵌入向量。

```
typedef int (*LLMGetEmbedCallback)(void* userdata, int32_t* tokens, uint64_t
num_tokens, void* embed, uint64_t len)
```

参数：

名称	说明
`userdata`	用户自定义数据指针，在注册回调时传入
`tokens`	token ID 数组
`num_tokens`	token 数组中的 token 数量
`embed`	指向用于存储嵌入输出的缓冲区的指针
`len`	嵌入缓冲区的长度（`字节`）

返回值：成功返回 0，失败返回非 0

使用说明：实现该回调时，应先确认目标缓冲区长度与 `num_tokens * embedding_dim * sizeof(float16)` 一致，再根据 token ID 从 `[vocab_size, embedding_dim]` 的 FP16 embedding table 查表并写入 Runtime 提供的目标缓冲区。Token ID 应位于有效词表范围内。

4.8.4 LLMTokenizerCallback

分词器回调函数类型，用于将输入文本转换为 token ID 序列。

```
typedef int (*LLMTokenizerCallback)(void* userdata, const char* text, int32_t
text_len, int32_t* tokens, int32_t n_tokens_max)
```

参数：

名称	说明
`userdata`	用户自定义数据指针，在注册回调时传入
`text`	指向输入文本字符串的指针
`text_len`	输入文本的长度
`tokens`	指向 token ID 数组的指针，用于存储分词结果
`n_tokens_max`	最大可生成的 token 数量

返回值：成功时返回实际写入的 token 数量，失败时返回负数错误码

使用说明：回调负责把 `text/text_len` 转成 token ID，并最多写入 `n_tokens_max` 个元素。应用可使用 HuggingFace `AutoTokenizer` 或自定义 Tokenizer 完成转换，最后返回实际写入 token 数；返回负值表示处理失败。

4.8.5 LLMOutputCallback

输出回调函数类型，用于在 LLM 推理过程中获取模型的输出张量。

```
typedef int (*LLMOutputCallback)(void* userdata, RKNN3Tensor* output_tensors,
uint32_t n_output_tensors, LLMOutputCallbackState state)
```

参数：

名称	说明
`userdata`	用户自定义数据指针，在注册回调时传入
`output_tensors`	指向输出张量数组的指针（`RKNN3Tensor`）
`n_output_tensors`	输出张量的数量
`state`	输出回调状态（LLMOutputCallbackState），如 PREFILL 处理中、PREFILL 完成、DECODE 处理中、DECODE 完成

返回值：成功返回 0，失败返回非 0

使用说明：

- 需要先为希望观察的输出准备 `RKNN3Tensor` 及其内存，并写入 `RKLLMCallback.output_tensors/n_output_tensors`；
- 回调中可通过 `tensor.attr` 读取输出元信息，通过 `tensor.mem.virt_addr` 读取实际数据；
- `state` 用于区分 Prefill 和 Decode 的处理阶段。应用只需要某一阶段结果时，应先判断 `state` 再读取输出，例如只在 `PREFILL_FINISHED` 时获取 Prefill embedding；
- 如果 Output Callback 的输出内存由应用显式创建，则退出前需要逐个调用 `destroy_mem()`。

4.8.6 LLMInputCallback

输入回调函数类型：

```
int (*LLMInputCallback)(void* userdata, RKNN3Tensor* tensors, uint32_t n_tensors,
LLMInputCallbackParam param)
```

该回调用于在 LLM 推理过程中由用户填充或更新指定输入张量，通常与 `input_tensors_index` 和 `n_input_tensors` 配合使用。

使用说明：

- `input_tensors_index` 指定由应用负责填充的模型输入索引，`n_input_tensors` 指定索引数量；
- 回调触发时，`tensors` 只包含本次需要应用处理的输入张量，可通过 `tensor.attr` 获取输入名称、Shape、dtype，通过 `tensor.mem` 获取目标缓冲区；
- `param.tokens`、`param.num_tokens` 和 `param.pos` 可用于根据当前 token 和上下文位置动态生成输入；
- 该机制适用于 per-layer embedding、Host RoPE Cache 等需要在推理期间按 token/位置更新额外输入的模型；
- 具体扩展输入名称、数据布局及填充算法属于模型约定，应用应根据模型导出结果和输入属性进行处理。

4.8.7 RKLLMCallback

LLM 回调函数结构：

字段	类型	说明
`result_callback`	`LLMResultCallback`	结果回调
`result_userdata`	`void*`	结果用户数据
`sampling_callback`	`LLMSamplingCallback`	采样回调
`sampling_userdata`	`void*`	采样用户数据
`tokenizer_callback`	`LLMTokenizerCallback`	分词器回调
`tokenizer_userdata`	`void*`	分词器用户数据
`embed_callback`	`LLMGetEmbedCallback`	嵌入回调
`embed_userdata`	`void*`	嵌入用户数据
`output_callback`	`LLMOutputCallback`	输出回调
`output_userdata`	`void*`	输出用户数据
`output_tensors`	`RKNN3Tensor*`	输出张量
`n_output_tensors`	`uint32`	输出张量数量
`input_callback`	`LLMInputCallback`	输入回调
`input_userdata`	`void*`	输入回调用户数据
`input_tensors_index`	`int32*`	输入张量索引
`n_input_tensors`	`uint32`	输入张量数量

使用说明：

- 常规 LLM 场景通常至少注册 `result_callback`、`tokenizer_callback`、`embed_callback`；
- Input Callback 除设置 `input_callback/input_userdata` 外，还需要设置 `input_tensors_index/n_input_tensors`；
- Output Callback 除设置 `output_callback/output_userdata` 外，还需要提供 `output_tensors/n_output_tensors`；
- `ctypes` Callback 对象、`ctypes.py_object` userdata、输入索引数组、输出张量数组以及回调内部引用的 NumPy/mmap 等对象，都必须在 Session 和回调使用期间保持 Python 引用；
- 无论同步还是异步 Session 场景，都不应依赖局部变量恰好未被回收。较稳妥的做法是把这些对象保存为长生命周期变量、对象成员或专门的 keepalive 列表。

4.8.8 LLMInputCallbackParam

LLM 输入回调参数，用于描述当前一次 Input Callback 对应的 token、Shape 和上下文位置信息。

名称	说明
`tokens`	当前回调对应的 token ID 数组；
`num_tokens`	本次需要处理的 token 数量；
`shape_id`	当前运行使用的 Shape ID；
`pos`	本次 token 在当前上下文中的起始位置；
`mrope_pos`	与 M-RoPE 位置相关的信息；
`mrope_start`	M-RoPE 起始位置；
`system_prompt_seqlen`	系统 Prompt 序列长度；
`valid_system_prompt_seqlen`	当前有效的系统 Prompt 序列长度；
`max_position_embeddings`	模型支持的最大位置编码长度；
`reserved`	保留字段。

使用说明：

- 对于只需要根据 token 查表的扩展输入，可主要使用 `tokens` 和 `num_tokens`；
- 对于需要按上下文位置切片的数据，可结合 `pos` 计算当前应填入的数据区域；
- M-RoPE 和系统 Prompt 相关字段是否需要使用取决于具体模型；
- 回调参数由 Runtime 提供，应用只读取其值，不应保存其中的 Runtime 指针并在回调结束后继续访问。

4.9 包装类

包装类用于把 Python 侧 `numpy.ndarray` 等对象转换成 RKNN3Lite 高层接口能够识别的辅助输入描述。DeepStack VLM 是其典型使用场景。

4.9.1 RKNN3AuxTensorWrapper

辅助张量包装类：

字段	类型	说明
`aux_data`	`numpy数组`	辅助数据
`index`	`int`	索引
`align_size`	`int`	对齐大小

使用说明：DeepStack VLM 场景可将视觉模型的额外输出包装成 AUX 输入：

```
aux = RKNN3AuxTensorWrapper()
aux.index = llm_input_index
aux.aux_data = vision_outputs[i]
aux.align_size = aligned_size
inputs.append(aux)
```

`index` 应对应 LLM 模型中实际的 AUX 输入位置，建议先通过张量属性查询确认，而不要假定固定索引；`align_size` 应与该输入的对齐要求一致。

5. 公共工具函数

5.1 rknn_dtype_to_numpy_dtype

API	`rknn_dtype_to_numpy_dtype`
功能	将 RKNN3 张量数据类型枚举转换为对应的 NumPy 数据类型。该函数用于在 Python 环境中处理张量数据时，确保数据类型的正确映射。
参数	`dtype_enum`：RKNN3 张量类型枚举值（RKNN3TensorType）
返回值	对应的 NumPy 数据类型（numpy.dtype）。如果输入的枚举值未找到映射，则返回 numpy.float32。

示例：

```
from rknn3lite.api.rknn3_types import rknn_dtype_to_numpy_dtype, RKNN3TensorType

# 转换 FP16 类型
np_dtype = rknn_dtype_to_numpy_dtype(RKNN3TensorType.RKNN3_TENSOR_FLOAT16)
print(np_dtype)  # 输出: <class 'numpy.float16'>
```

5.2 rknn3_get_layout_string

API	`rknn3_get_layout_string`
功能	根据张量布局枚举值获取对应的布局字符串表示。该函数用于调试和日志输出，帮助用户理解张量的布局格式。
参数	`layout`：RKNN3 张量布局枚举值（RKNN3TensorLayout）
返回值	布局的字符串表示（如 "NCHW", "NHWC"）。如果枚举值无效，则返回 "UNKNOWN"。

示例：

```
from rknn3lite.api.rknn3_types import rknn3_get_layout_string, RKNN3TensorLayout

# 获取 NHWC 布局字符串
layout_str = rknn3_get_layout_string(RKNN3TensorLayout.RKNN3_TENSOR_NHWC)
print(layout_str)  # 输出: NHWC
```

5.3 rknn3_get_type_string

API	`rknn3_get_type_string`
功能	根据张量数据类型枚举值获取对应的类型字符串表示。该函数用于调试和日志输出，帮助用户理解张量的数据类型。
参数	`dtype`：RKNN3 张量数据类型枚举值（RKNN3TensorType）
返回值	数据类型的字符串表示（如 "FP32", "INT8"）。如果枚举值无效，则返回 "UNKNOWN"。

示例：

```
from rknn3lite.api.rknn3_types import rknn3_get_type_string, RKNN3TensorType

# 获取 INT8 类型字符串
type_str = rknn3_get_type_string(RKNN3TensorType.RKNN3_TENSOR_INT8)
print(type_str)  # 输出: INT8
```

5.4 rknn3_get_qnt_type_string

API	`rknn3_get_qnt_type_string`
功能	根据张量量化类型枚举值获取对应的量化类型字符串表示，如 `NONE`、`PER_LAYER_SYMMETRIC`、`PER_CHANNEL_ASYMMETRIC` 等。若枚举值无效，则返回 `UNKNOWN`。
参数	`qnt_type`：RKNN3 张量量化类型枚举值（RKNN3TensorQntType）
返回值	量化类型字符串。

5.5 dump_tensor_attr

API	`dump_tensor_attr`
功能	将张量属性的详细信息打印到控制台。该函数用于调试目的，帮助开发者检查张量的各种属性信息，包括形状、数据类型、布局等。
参数	`attr`：RKNN3 张量属性结构（RKNN3TensorAttr）
	`prefix`：打印前缀字符串，默认为 "tensor"
返回值	无

示例：

```
from rknn3lite.api.rknn3_types import dump_tensor_attr

# 假设有张量属性对象 attr
dump_tensor_attr(attr, prefix="input")
# 输出示例:
# input_0: name=input_tensor, n_dims=4, shape=[1, 3, 224, 224], layout=NHWC,
dtype=FP32, ...
```

5.6 get_os_platform

API	`get_os_platform`
功能	获取当前操作系统的平台字符串。该函数用于确定运行环境的架构（32位或64位），有助于进行平台相关的配置和兼容性检查。
参数	无
返回值	平台字符串，格式为 "{操作系统}_{架构}"，如 "Linux_x64" 或 "Windows_x32"。

示例：

```
from rknn3lite.api.rknn3_types import get_os_platform

platform = get_os_platform()
print(platform)  # 输出: Linux_x64 (在64位Linux系统上)
```

6. 异常与错误处理

RKNN3 Toolkit Lite 的 Python API 主要通过**返回值和日志**反馈运行错误。部分接口在参数检查或调用底层 Runtime 失败时返回 `-1`，查询类或对象类接口通常返回 `None`；底层 RKNN3 Runtime 的具体错误原因可结合 4.1.2 状态码 判断。

除 Runtime 错误外，Python 参数处理、文件访问、NumPy、Tokenizer、`ctypes` Callback 以及用户自定义逻辑仍可能产生 Python 异常。应用应同时检查接口返回值和日志，不应只依赖 `try/except` 判断 RKNN3 Runtime 是否执行成功。

6.1 错误返回与日志

RKNN3 Toolkit Lite 不要求所有接口使用完全相同的失败返回形式。常见情况如下：

接口类型	成功返回	常见失败返回
状态型接口	`0`	`-1` 或底层非 0 返回值
Query/对象接口	查询结果或对象	`None`
列表接口	列表	`None` 或空结果
LoRA 查询	`(lora_list, n_lora)`	`(None, 0)`
Context 复制	`(ret, new_context)`	`(-1, None)`
LLM 推理	`(ret, stats)`	通常返回 `(-1, [])`
图像内存创建	`(ret, im_mem)`	失败结果以接口返回值为准

因此，调用方应按照第 3 章每个 API 的“返回值”说明分别判断。例如：

```
ret = rknn.init_runtime(target="rk1820", core_mask=0x01)
if ret != 0:
    print(f"init_runtime failed, ret={ret}")
    rknn.release()
    return
```

查询类接口应检查 `None`：

```
io_num = rknn.rknn3_query(RKNN3QueryCmd.RKNN3_QUERY_IN_OUT_NUM)
if io_num is None:
    print("query input/output number failed")
    rknn.release()
    return
```

对于 LLM 推理，`session_run()` / `session_run_async()` 在未初始化或捕获异常时通常返回 `(-1, [])`，应直接检查返回码：

```
ret, stats = rknn.session_run(prompt="hello")
if ret != 0:
    print(f"session_run failed, ret={ret}")
```

创建 `RKNN3Lite(verbose=True)` 可以输出更详细的运行日志；也可以通过 `verbose_file` 将日志保存到文件：

```
rknn = RKNN3Lite(
    llm_mode=True,
    verbose=True,
    verbose_file="./rknn3lite.log"
)
```

出现问题时，建议同时保存 Python 侧日志和底层 Runtime 返回码，以便定位错误发生在参数检查、Python 封装还是设备 Runtime。

6.2 Runtime 和模型初始化错误

Runtime 和模型初始化是后续所有推理操作的前置条件。常见问题如下：

现象	可能原因	处理方法
`load_rknn()` 返回失败	模型路径或权重路径不存在/无效	检查 `.rknn` 与 `.weight` 文件路径是否存在且有效
`init_runtime()` 返回失败	可能包括模型与权重不匹配、模型格式异常、平台不匹配等	检查模型与权重是否配套，确认模型格式与目标平台匹配
`init_runtime()` 返回失败	target、core_mask、device_id、模型、Runtime 或设备状态异常	检查参数，并结合 4.1.2 状态码和 Runtime 日志定位
`RKNN3_ERR_MODEL_INVALID`	模型文件无效或无法被当前 Runtime 正确加载	确认模型文件完整，并确认模型与当前 SDK/Runtime 配套
`RKNN3_ERR_TARGET_PLATFORM_UNMATCH`	模型目标平台与实际运行平台不匹配	使用与目标 NPU 平台匹配的 RKNN 模型
`RKNN3_ERR_DEVICE_UNAVAILABLE`	目标设备不可用	使用 `get_devices_id()` 检查设备是否存在，并检查设备侧服务
`RKNN3_ERR_INCOMPATIBLE_VERSION`	Runtime 相关组件版本不一致	确认 Python Wheel、Runtime 动态库及设备侧组件使用配套版本
`RKNN3_ERR_KEY_INVALID` / `RKNN3_ERR_DECRYPT_FAILED`	解密密钥无效、密钥与模型不匹配或加密数据损坏	检查 `decrypt_key_path` / `decrypt_key_data` 以及加密模型文件
`RKNN3_WARN_NPU_CORE_UNUSED`	指定的部分 NPU Core 未参与模型执行	该状态属于警告；如执行结果正常，可以继续运行

`core_mask` 是初始化阶段常见的配置错误来源。建议先根据模型和设备支持的 Core 数量确认掩码，不要假定所有模型都应该使用全部 NPU Core。

如果 `init_runtime()` 已经失败，不应继续调用 `inference()`、`rknn3_query()`、`init_llm_session()` 等依赖 Runtime 的接口，应先处理初始化错误并重新创建或初始化 Runtime。

6.3 Query、输入和推理错误

Query 和推理错误通常与调用顺序、输入数量、Shape、dtype 或 Layout 不匹配有关。

6.3.1 Query 失败

`rknn3_query()` 失败时通常返回 `None`。常见原因包括：

- Runtime 尚未初始化；
- Query Cmd 不支持；

- 输入/输出索引超出模型实际范围；
- 当前模型不支持对应查询项；
- 底层 Runtime Query 返回错误。

推荐先查询输入输出数量，再按合法索引查询 Tensor 属性：

```
io_num = rknn.rknn3_query(RKNN3QueryCmd.RKNN3_QUERY_IN_OUT_NUM)
if io_num is None:
    return

for i in range(io_num.n_input):
    attr = rknn.rknn3_query(
        RKNN3QueryCmd.RKNN3_QUERY_INPUT_ATTR,
        index=i
    )
    if attr is None:
        print(f"query input attr[{i}] failed")
        return
```

6.3.2 通用模型输入错误

`inference()` 前应确认：

- `inputs` 数量与模型输入数量一致；
- NumPy Shape 与模型当前 Shape 匹配；
- dtype 可以正确转换为模型要求的数据类型；
- 4 维输入使用的 `data_format` 与实际逻辑布局一致；
- 动态 Shape 输入属于模型支持的 Shape 组合。

出现 `RKNN3_ERR_INPUT_INVALID` 时，应优先检查上述输入信息；出现 `RKNN3_ERR_OUTPUT_INVALID` 时，应检查输出 Tensor 配置和显式输出内存。

动态 Shape 模型可以通过：

```
shape_infos = rknn.get_dynamic_shape_infos()
```

查看支持的 Shape 组合；需要显式切换时使用：

```
ret = rknn.set_shape(shape_id)
```

6.4 LLM Session 和输入错误

LLM/VLM 接口除依赖 Runtime 外，还依赖已经初始化的 LLM Session。

如果在 LLM Runtime 或 Session 尚未初始化时调用 Session、KVCache、LoRA、Chat Template 等接口，Python 层通常记录类似“LLM runtime environment is not inited”的错误并返回 `-1` 或 `None`。

推荐遵循以下顺序：

```
RKNN3Lite(llm_mode=True)
    ↓
load_rknn()
    ↓
init_runtime()
    ↓
init_llm_session()
    ↓
session_run()
```

6.4.1 Session 索引错误

多 Session 使用连续的 `session_index`。创建新 Session 时：

- `session_index < 0`：无效；
- `session_index < 当前 Session 数量`：该索引已经初始化；
- `session_index > 当前 Session 数量`：跳过了前面的 Session；
- 正确的新 Session 索引应等于当前 Session 数量。

例如当前只有 Session 0 时，应先创建 Session 1，不能直接创建 Session 2。

出现 Session 索引错误时，可以通过：

```
session_count = rknn.get_session_count()
```

获取当前已初始化 Session 数量，再决定新的 `session_index`。

6.4.2 LLM 主输入互斥

`session_run()` 和 `session_run_async()` 的：

- `prompt`
- `embeds`
- `tokens`

三种主输入互斥，一次调用只能选择其中一种。

错误：

```
rknn.session_run(
    prompt="hello",
    tokens=[1, 2, 3]
)
```

正确：

```
rknn.session_run(prompt="hello")
```

或者：

```
rknn.session_run(tokens=[1, 2, 3])
```

`inputs` 用于 `RKNN3Image`、`RKNN3Audio`、`RKNN3Video` 或 AUX 等多模态/辅助输入，可以按模型要求与其中一种主输入共同使用。

6.4.3 上下文长度错误

LLM 的 `max_context_len` 不应超过模型 `RKNN3LLMConfig.max_ctx_len`。建议在创建 Session 前先查询：

```
llm_config = rknn.rknn3_query(
    RKNN3QueryCmd.RKNN3_QUERY_LLM_CONFIG
)
```

然后以：

```
llm_config.max_ctx_len
```

作为应用配置上限。

运行过程中还可以通过 `session_query_state()` 检查：

- `n_total_tokens`
- `n_max_tokens`

当上下文接近上限时，可根据业务需要清理 KVCache 或结束当前会话。

6.5 KVCache 和 LoRA 错误

6.5.1 KVCache

KVCache 接口只能在有效 LLM Session 上使用。常见问题包括：

- Session 未初始化；
- `session_index` 无效；
- KVCache 文件不存在或与当前模型/Session 配置不匹配；
- 内存加载时 `kvcache_data` 与 `size` 不一致；
- `group_id` 超出 `RKNN3_QUERY_KVCACHE_LEN_GROUP_INFO` 返回的有效范围；
- 使用外部 KVCache 内存时，目标分组需要的内存大于已提供内存。

调用 `set_kvcache_group()` 前建议先检查：

```
infos = rknn.rknn3_query(
    RKNN3QueryCmd.RKNN3_QUERY_KVCACHE_LEN_GROUP_INFO
)

if infos is not None:
    n_groups = infos[0].n_groups
    if 0 <= group_id < n_groups:
        rknn.set_kvcache_group(group_id)
```

分页 KV Cache 使用 `set_kvcache_pages()`，常见问题包括：

- 对非分页模型调用该接口。应先通过 `RKNN3_QUERY_KVCACHE_PAGE_INFO` 确认 `n_cache_types` 不为 0，否则应改用 `set_kvcache_mem()`；
- `bindings` 中条目的 `core_id`、`cache_type` 与模型实际配置不匹配，导致形状校验失败；
- `n_pages` 超过 `RKNN3KvCachePageInfo.caches[i].max_page_num`；
- 单个 page 的大小不是 `RKNN3CachePageAttr.page_size`，或未按 `core_id` 指定的 Core 分配；
- `pages` 指向的指针数组在调用期间被回收，或在解绑前提前释放了 page；
- 在 Context 非空闲（例如推理进行中）时调用该接口。

由于 `set_kvcache_pages()` 是 Context 级操作，其错误处理与 Session 无关，应结合 `RKNN3_QUERY_KVCACHE_PAGE_INFO` 和 `RKNN3_QUERY_ATTENTION_CONFIG` 的查询结果排查配置是否一致。

6.5.2 LoRA

LoRA 应按照以下顺序使用：

```
init_lora()
    ↓
query_lora()
    ↓
load_lora()
    ↓
enable_lora()
```

常见错误包括：

- LLM Runtime 尚未初始化；
- LoRA 权重路径或内存数据无效；
- 使用了不存在的 `lora_name`；
- 尚未 `load_lora()` 就调用 `enable_lora()`；
- 对已经释放或不存在的 Session 操作 LoRA；
- LoRA 与 Base 模型不匹配。

如果需要确认可用适配器，应优先使用 `query_lora()` 返回的 `RKNN3Lora` 信息，而不是在应用中硬编码适配器名称。

6.6 内存相关错误

内存相关问题通常表现为创建内存失败、同步失败、NPU 运行失败或输出数据异常。

6.6.1 内存不足

底层返回：

```
RKNN3_ERR_OUT_OF_MEMORY
```

时，表示当前设备或指定 NPU Core 无法满足内存需求。

可以使用：

```
mem_info = rknn.rknn3_query(
    RKNN3QueryCmd.RKNN3_QUERY_DEVICE_MEM_INFO
)
```

检查系统总内存、可用内存以及各 NPU Node 的内存情况。

对于 LLM，还应关注：

- `max_context_len`

- KVCache 长度分组；
- 用户自定义 KVCache 内存；
- 同时存在的 Session 数量；
- LoRA 和额外 Output Tensor 的内存占用。

6.6.2 内存同步失败

`RKNN3_ERR_MEM_SYNC_FAILED` 表示 CPU 与设备之间的内存同步失败。

使用显式 Tensor Memory 时应注意：

- CPU 写入、NPU 读取前使用 `RKNN3_MEMORY_SYNC_TO_DEVICE`；
- NPU 写入、CPU 读取前使用 `RKNN3_MEMORY_SYNC_FROM_DEVICE`；
- `mem_sync_range()` 必须满足 `offset + length <= mem.size`；
- `length`、`offset` 应按字节计算；
- 已经 `destroy_mem()` 的内存不能再次同步或访问。

6.6.3 外部内存生命周期

`create_mem_from_phys()`、`create_mem_from_fd()` 和用户提供的 Runtime 内存依赖应用侧缓冲区。应用必须保证底层物理内存、FD、虚拟地址等在 RKNN3 使用期间始终有效。

`destroy_mem()` 用于释放 RKNN3 对该内存的关联对象；外部内存本身是否需要由应用继续释放，取决于其原始分配方式。

6.7 Callback 异常

LLM Callback 运行在底层 Runtime 调用链中。Callback 中发生未处理的 Python 异常时，可能导致本次回调失败或推理异常，因此建议在 Callback 内部自行捕获异常。

推荐形式：

```
def result_callback(userdata, result_ptr, state):
    try:
        # 处理结果
        return 0
    except Exception as e:
        print(f"result callback failed: {e}")
        return -1
```

Callback 常见问题如下：

问题	原因	处理方法
Callback 不再触发或发生非法访问	Callback Python 对象被垃圾回收	在 Session 生命周期内保存 Callback 对象引用
userdata 访问异常	`ctypes.py_object` 或其指针生命周期结束	保存 userdata 及其底层 Python 对象
Embedding Callback 返回失败	Buffer 长度或 token ID 不合法	检查 `num_tokens`、`embedding_dim`、`len` 和词表范围
Input Callback 写入错误	Tensor 名称、Shape、dtype、当前位置处理错误	通过 `tensor.attr` 和 `LLMInputCallbackParam` 校验目标缓冲区
Output Callback 数据异常	在错误的回调阶段读取，或 dtype 解释错误	根据 `LLMOutputCallbackState` 判断阶段，并按 Tensor attr 解释内存
流式文本出现 `�`	单个 token 不能构成完整 UTF-8 字符	累积 token 后统一 decode，或等待后续 token

以下对象应至少保持到 Session 不再使用对应 Callback 为止：

- `RKLLMCallback`
- 各 `ctypes.CFUNCTYPE` Callback 对象；
- `ctypes.py_object` userdata；
- `input_tensors_index`；
- `output_tensors`；
- Callback 直接引用的 NumPy、mmap 或其他外部缓冲区。

6.8 异步执行错误

通用模型和 LLM 的异步接口使用方式不同。

通用模型

调用顺序应为：

```
inference_async()
    ↓
wait()
    ↓
get_outputs()
```

在异步任务完成前，不应提前释放输入、Runtime 或相关显式内存。

LLM

`session_run_async()` 的返回值仅表示任务已经提交，并不表示生成已经结束。真正完成状态应通过 `LLMResultCallback` 判断。

常见终态：

- `RKLLM_RUN_FINISH`
- `RKLLM_RUN_STOP`
- `RKLLM_RUN_MAX_NEW_TOKEN_REACHED`
- `RKLLM_RUN_ERROR`

如果需要停止或暂停正在执行的请求，可以调用：

```
rknn.session_stop(session_index=0)
rknn.session_pause(session_index=0)
rknn.session_resume(session_index=0)
```

应用应协调同一 Session 上的运行、停止、暂停、恢复和释放操作，避免在任务仍然运行时释放 Session 或 Runtime。

6.9 错误定位建议

出现 RKNN3 Toolkit Lite 错误时，建议按照以下顺序排查：

1. **确认调用顺序** 检查是否已经完成 `load_rknn()`、`init_runtime()`；LLM 是否已经完成 `init_llm_session()`。
2. **检查 Python API 返回值** 根据对应接口判断 `ret != 0`、`result is None` 或其他失败形式。

3. **查看详细日志** 使用 `verbose=True`，必要时通过 `verbose_file` 保存完整日志。
4. **确认底层状态码** 根据日志中的 Runtime error code 对照 4.1.2 状态码。
5. **检查模型和平台** 确认 RKNN 模型的目标平台、模型文件、Weight 文件和当前 NPU 平台匹配。
6. **检查版本** 如出现 `RKNN3_ERR_INCOMPATIBLE_VERSION`，检查 Python Wheel、Runtime 动态库以及设备侧相关组件版本。
7. **检查模型 I/O** 使用 `RKNN3_QUERY_IN_OUT_NUM`、`INPUT_ATTR`、`OUTPUT_ATTR` 确认输入输出数量、Shape、dtype 和 Layout。
8. **检查设备和内存** 使用 `get_devices_id()`、`RKNN3_QUERY_DEVICE_MEM_INFO` 和 `RKNN3_QUERY_CORE_NUMBER` 确认设备、Core 与可用内存。
9. **LLM 进一步检查 Session 状态** 使用 `get_session_count()`、`session_query_state()` 检查 Session、Token 和 KVCache 状态。
0. **确认资源生命周期** 检查 Tensor Memory、NumPy Buffer、Callback、userdata、Session 和 Runtime 是否在仍被使用时提前释放。

若错误发生在底层 Runtime 且 Python 日志无法进一步定位，应保留以下信息用于问题分析：

- RKNN3 Toolkit Lite/Python Wheel 版本；
- `get_sdk_version()` 返回信息；
- 硬件平台和 `device_id`；
- RKNN 模型与 Weight 文件信息；
- `target` 和 `core_mask`；
- 完整错误日志；
- Runtime 状态码；
- 能够复现问题的最小调用流程。